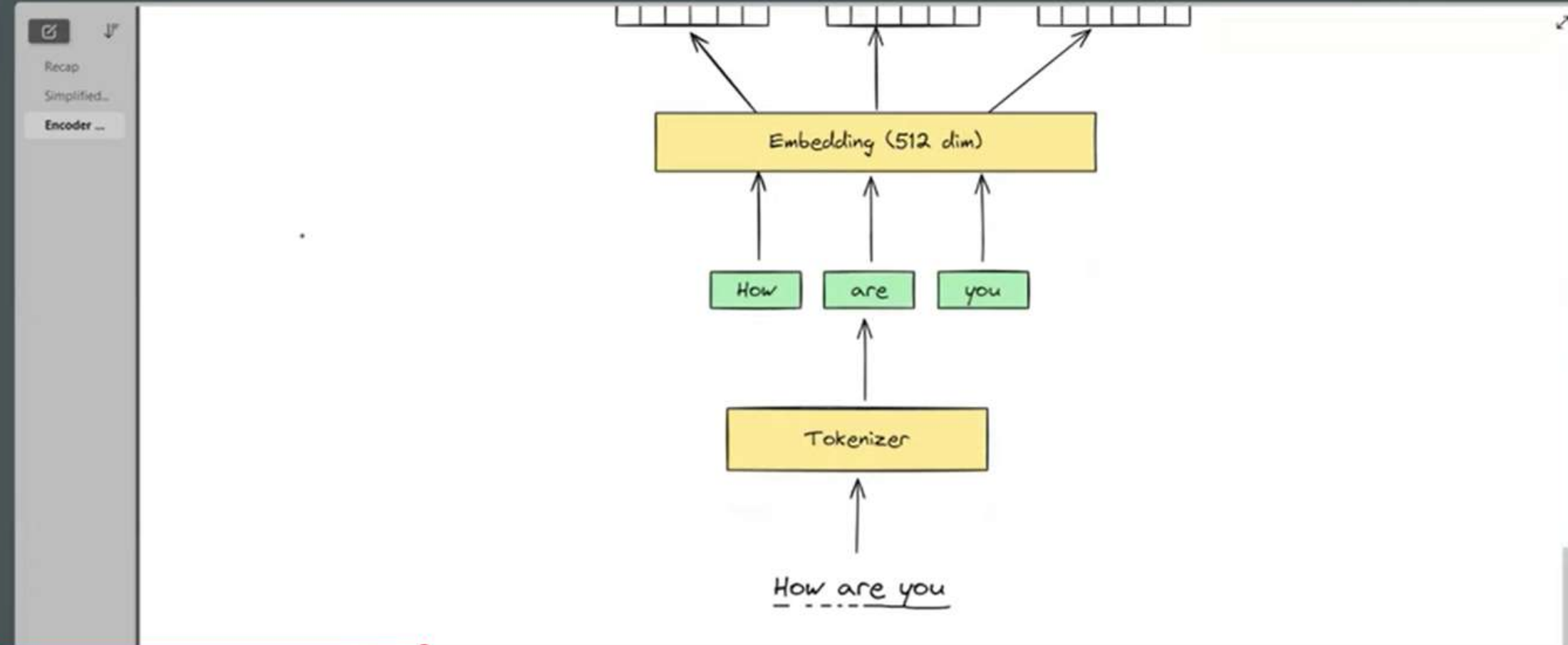
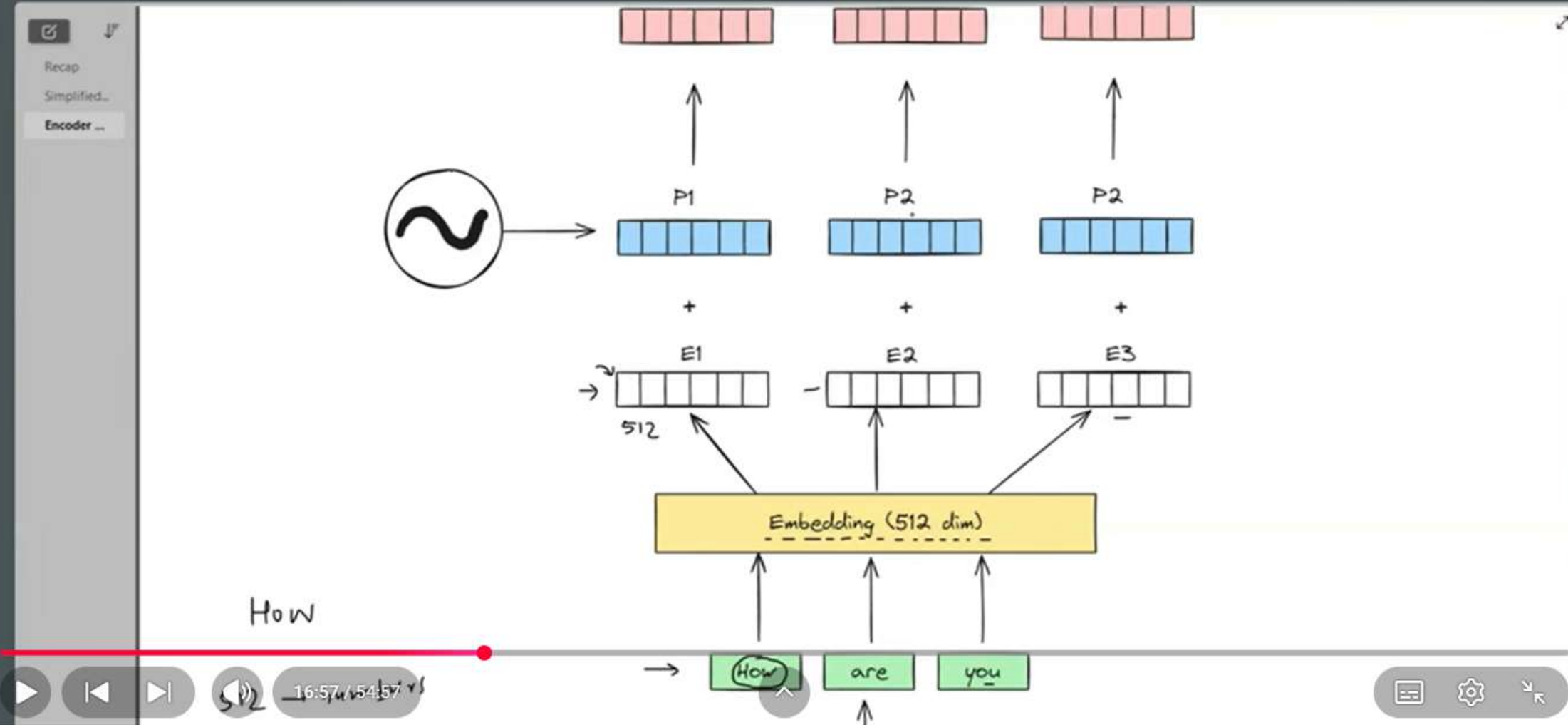
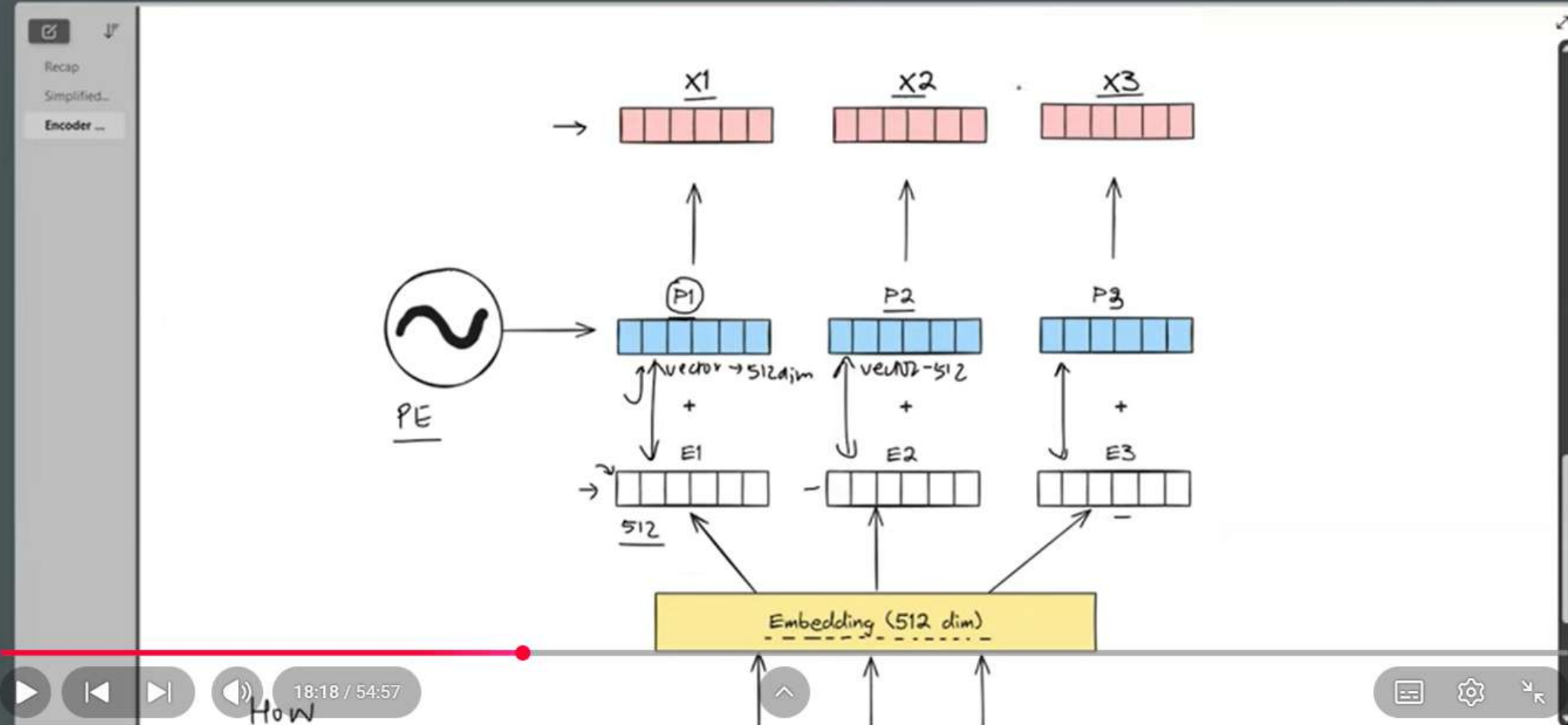
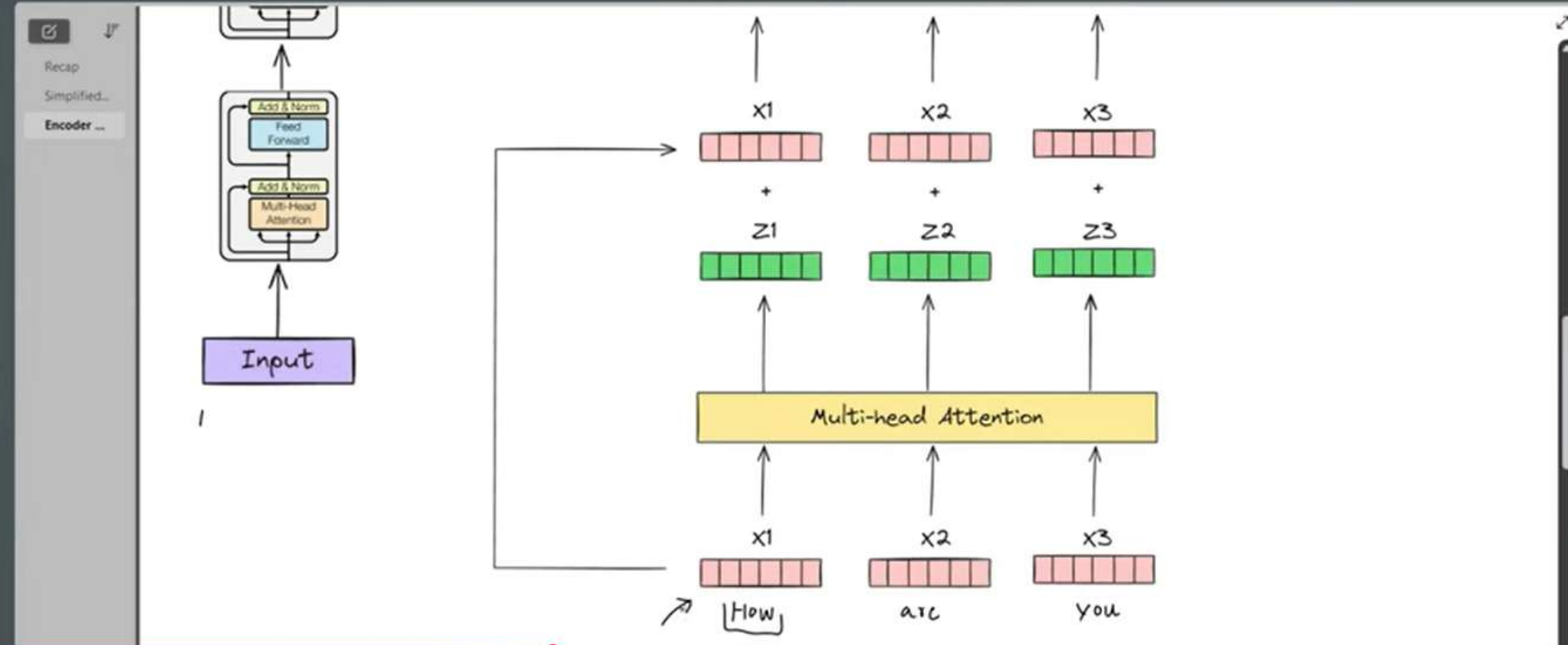
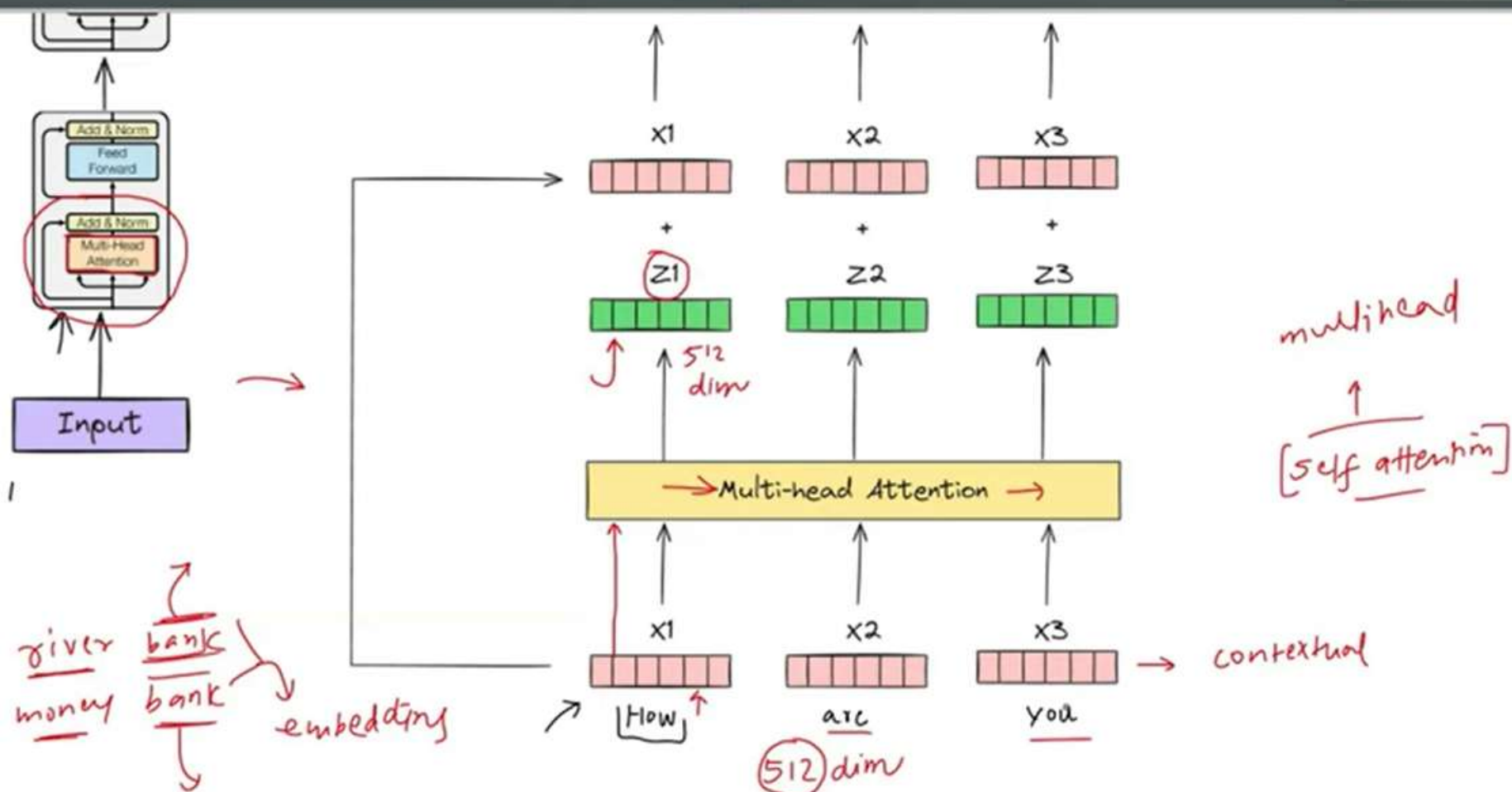


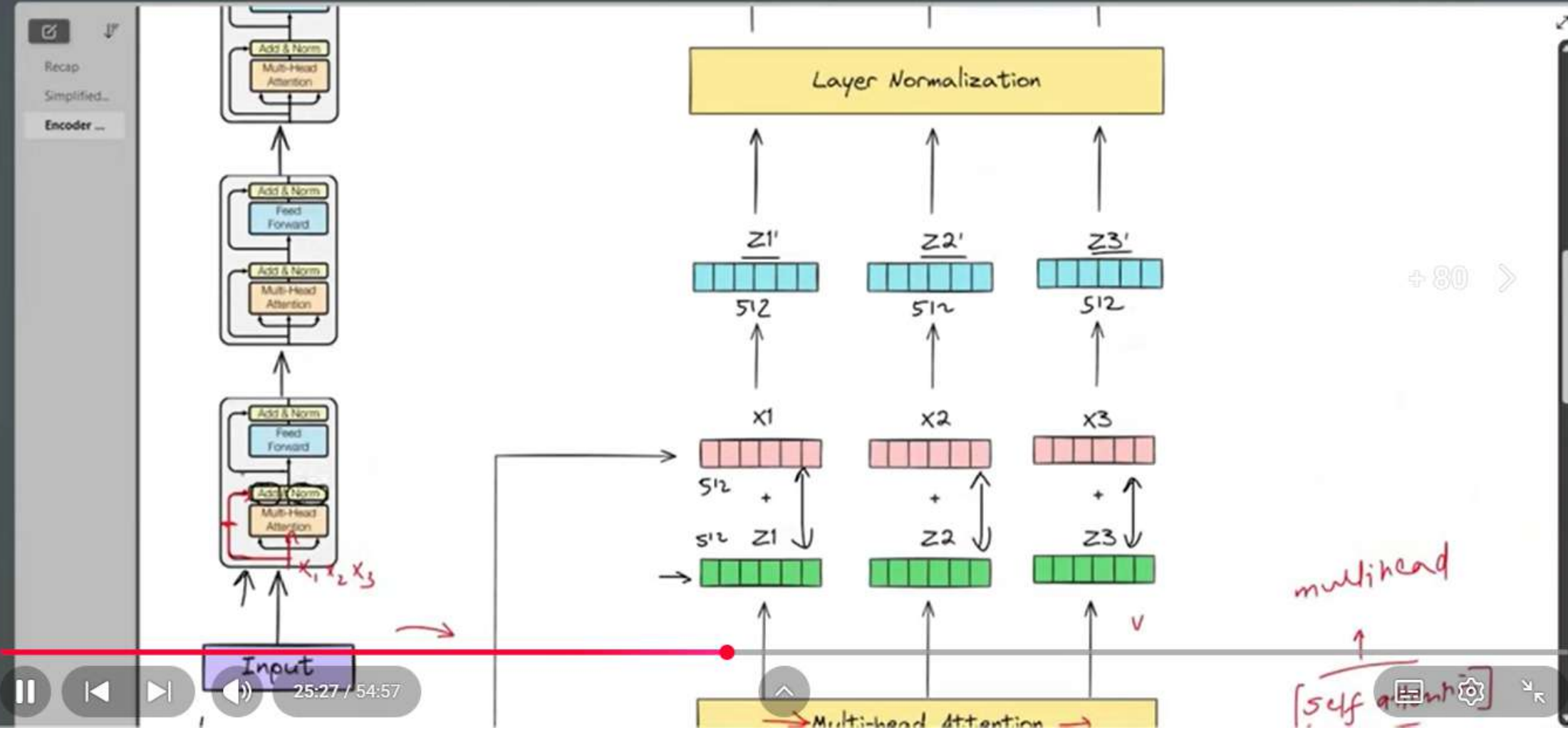


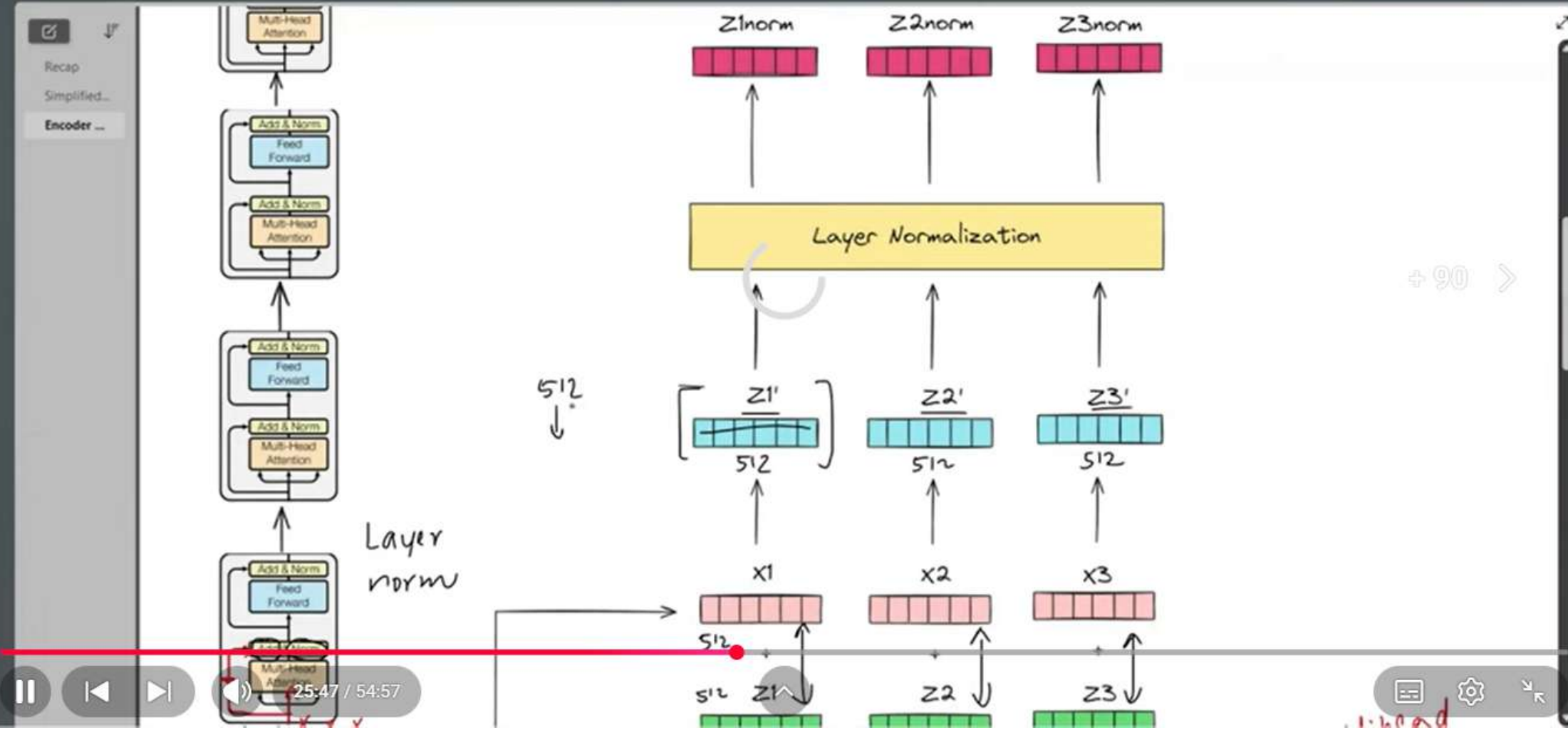


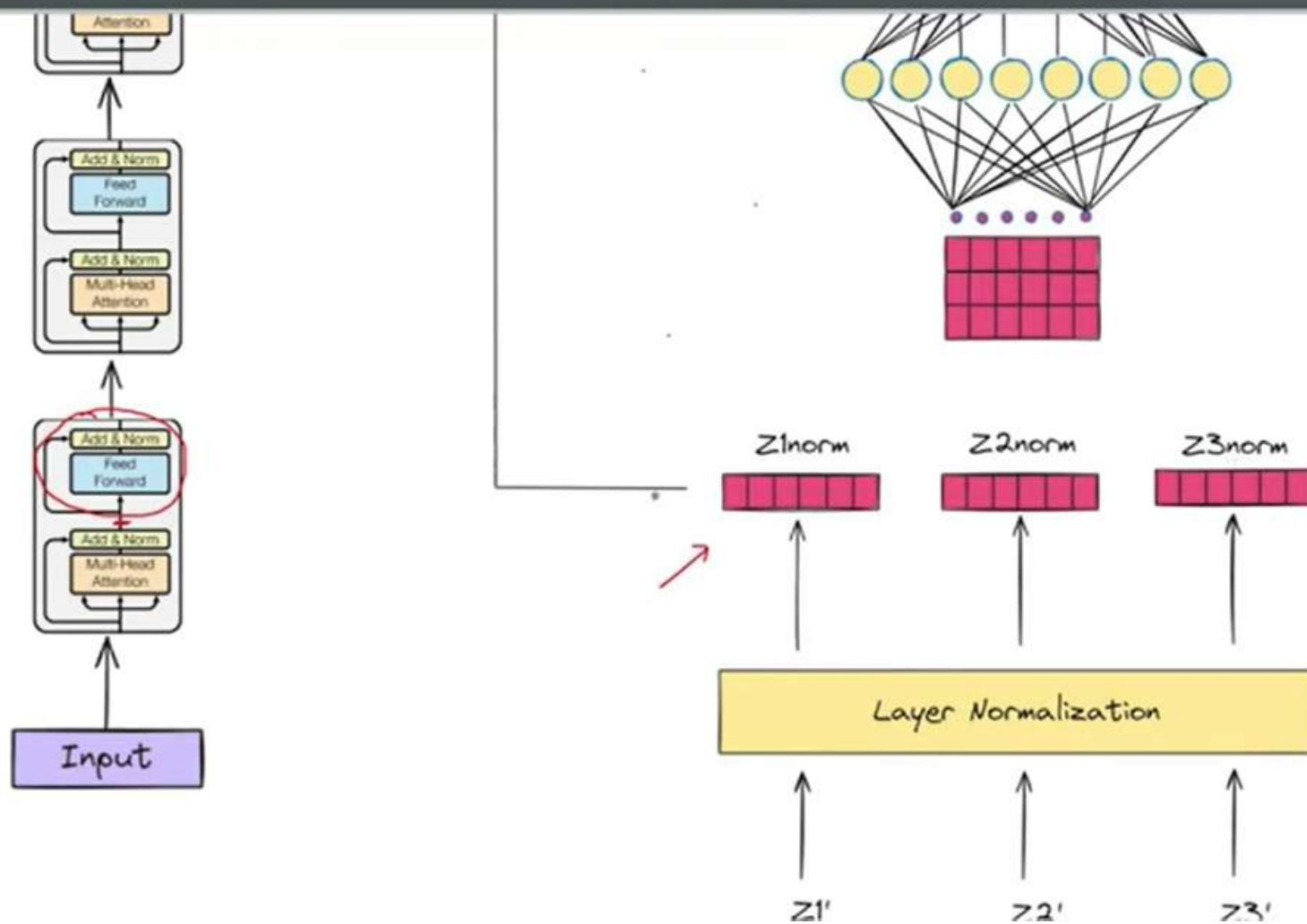


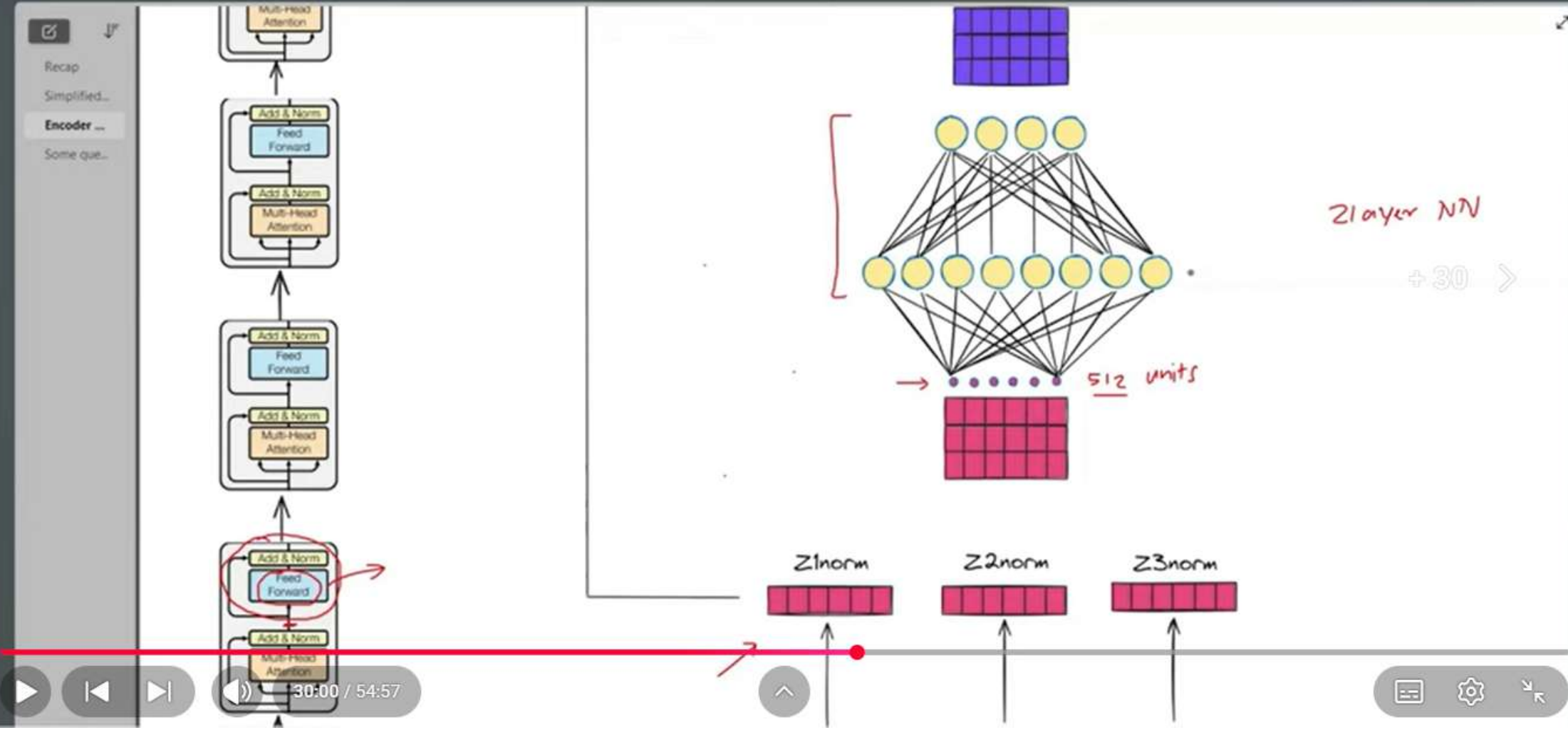


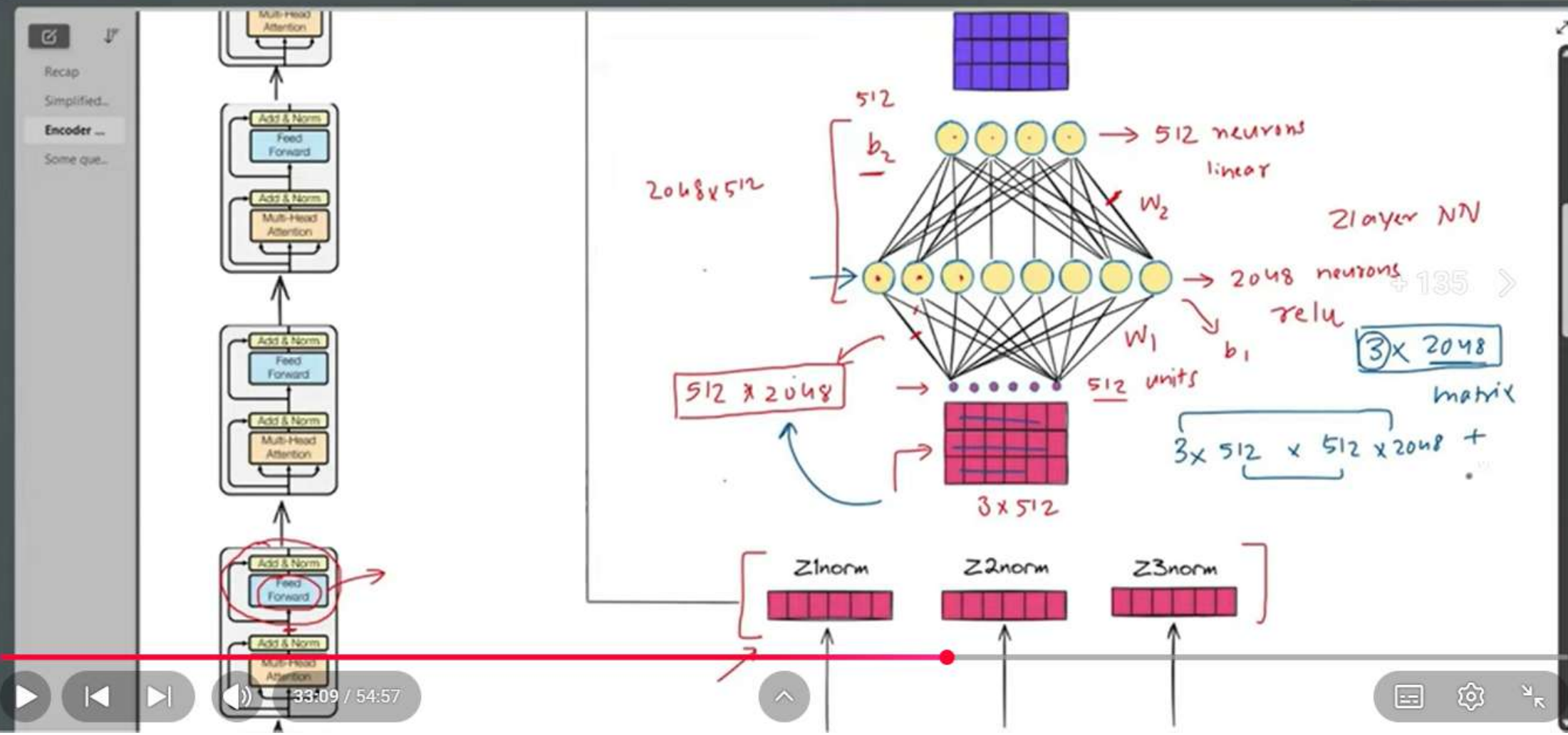


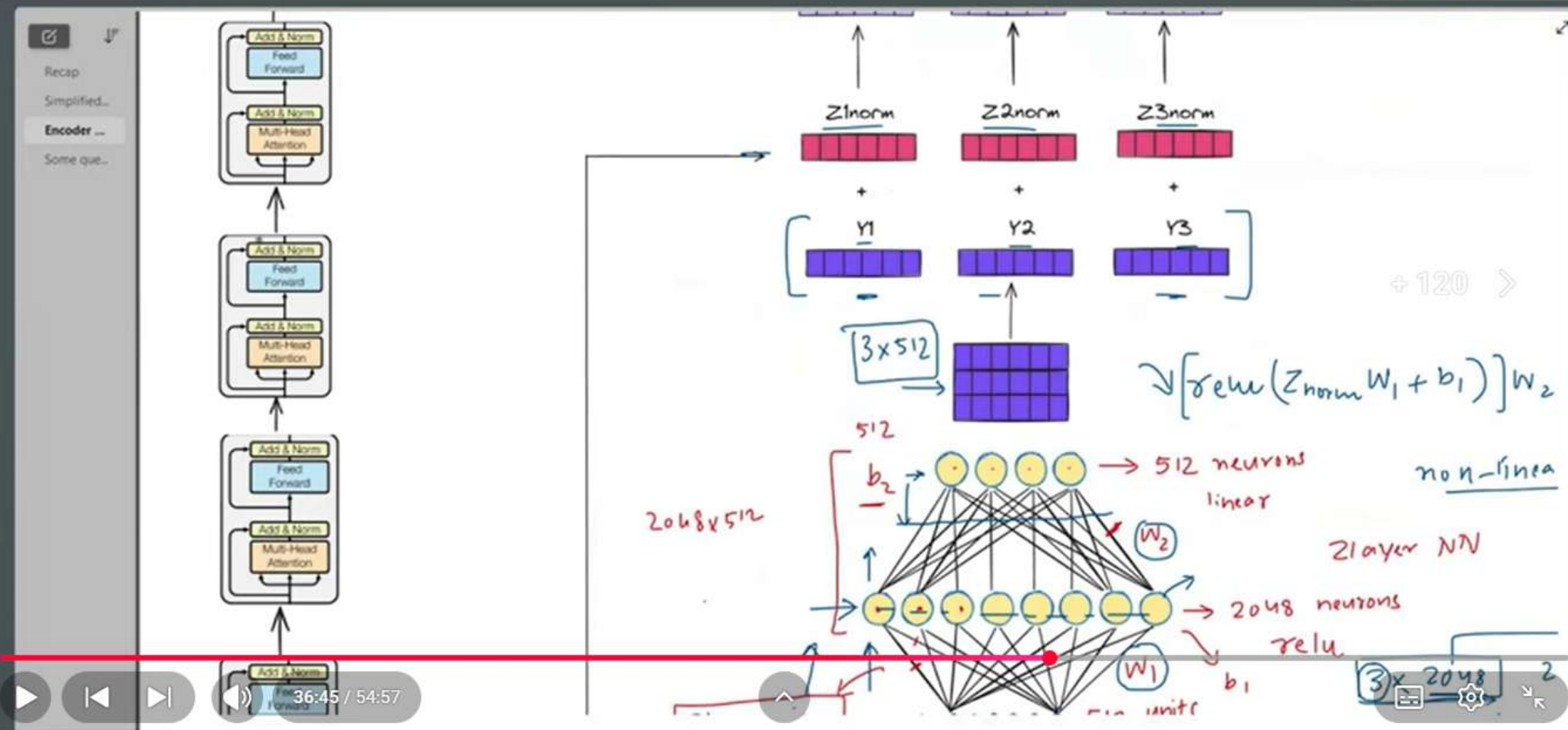








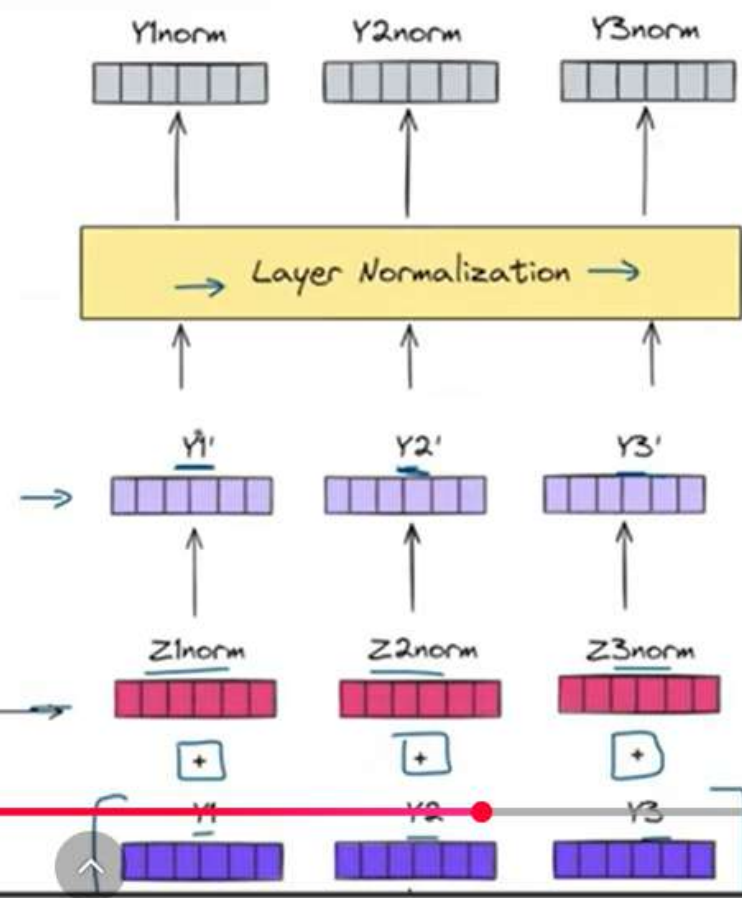
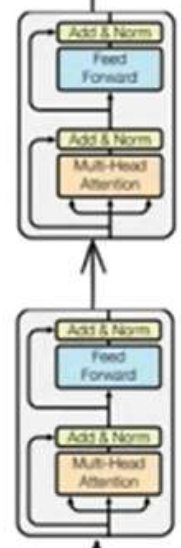




Encoder Architecture

09 July 2024 16:31

Output



Some questions

09 July 2024 20:48

1. Why use residual connections? → Kaggle → comment
2. Why use a FFNN?
3. Why use 6 encoder blocks?

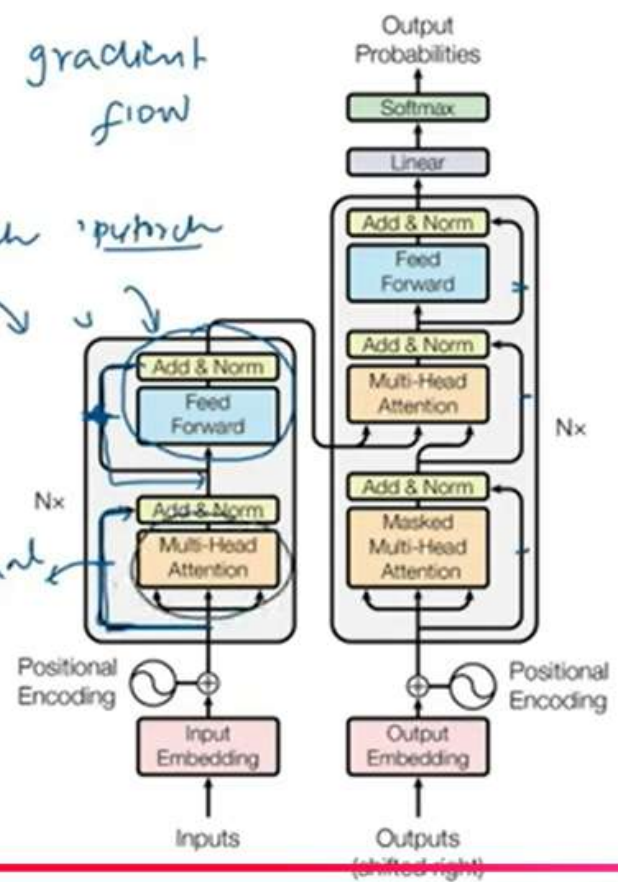
1) Stable training - deep NN
ResNet
vanishing Grad

Alternak.

gradient flow

search 'puthru'

Residual



+140 >

Transformer Feed-Forward Layers Are Key-Value Memories

Mor Geva^{1,2} Roel Schuster^{1,3} Jonathan Berant^{1,2} Omer Levy¹

¹Blavatnik School of Computer Science, Tel-Aviv University

²Allen Institute for Artificial Intelligence

³Cornell Tech

{morgeva@mail, jobarrant@cs, levyomer@cs}.tau.ac.il, rs864@cornell.edu

+ 10 >

Abstract

Feed-forward layers constitute two-thirds of a transformer model's parameters, yet their role in the network remains under-explored. We show that feed-forward layers in transformer-based language models operate as key-value memories, where each key correlates with textual patterns in the training examples, and each value induces a distribution over the output vocabulary. Our experiments show that the learned patterns are human-interpretable, and that lower layers tend to capture shallow patterns, while upper layers learn more semantic ones. The values complement the keys' input patterns by inducing output distributions that concentrate probability mass on tokens likely to appear immediately after each pattern, particularly in the upper layers. Finally, we demonstrate that the output of a feed-forward layer is a composition of its memories, which is subsequently refined throughout the model's layers via residual connections to produce the final output distribution.

In conclusion, our work sheds light on the function of feed-forward layers in transformer-based language models. We show that feed-forward layers act as pattern detectors over the input across all layers, and that the final output distribution is gradually constructed in a bottom-up fashion.¹

2 Feed-Forward Layers as Unnormalized Key-Value Memories

Feed-forward layers

A transformer language model Vaswani et al. (2017) is made of intertwined self-attention and feed-forward layers. Each feed-forward layer is a position-wise function, processing each input vector independently. Let $\mathbf{x} \in \mathbb{R}^d$ be a vector corresponding to some input text prefix. We can express the feed-forward layer $\text{FF}(\cdot)$ as follows (bias terms are omitted):

$$\text{FF}(\mathbf{x}) = f(\mathbf{x} \cdot K^T) \cdot V \quad (1)$$

Here, $K, V \in \mathbb{R}^{d_m \times d}$ are parameter matrices, and f is a non-linearity such as ReLU.

Neural memory

A neural memory Sukhbaatar et al. (2015) consists of d_m key-value pairs, which we call *memories*.² Each key is represented by a d -dimensional vector $\mathbf{k}_i \in \mathbb{R}^d$, and together form the parameter matrix $K \in \mathbb{R}^{d_m \times d}$; likewise, we define the value parameters as $V \in \mathbb{R}^{d_m \times d}$. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, we compute a distribution over the keys, and use it to compute the expected value:

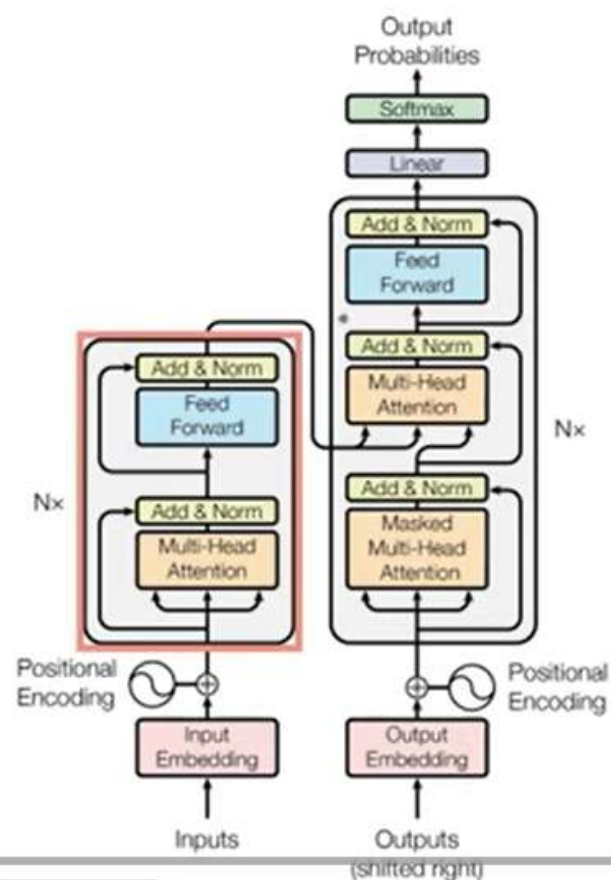
$$p(k_i | \mathbf{x}) \propto \exp(\mathbf{x} \cdot \mathbf{k}_i) \\ \text{MN}(\mathbf{x}) = \sum_{i=1}^{d_m} p(k_i | \mathbf{x}) \mathbf{v}_i$$

With matrix notation, we arrive at a more compact formulation:

$$\text{MN}(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot K^T) \cdot V \quad (2)$$

¹
The code for reproducing our experiments is available at <https://github.com/mega002/ff-layers/>.

²
We use the terms "memory cells" and "memories" interchangeably.



Autoregressive models - OneNote

Search

File Home Insert Draw History Review View Help

Sticky Notes Share

Search Notebooks

Add Page

Recap

Autoregressive models

Autoregressive models

23 July 2024 17:39

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.

Add Page

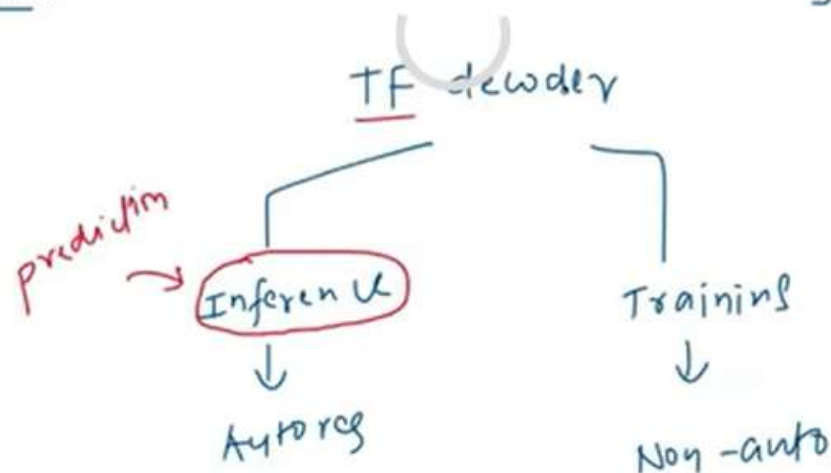
Recap

Autoregressive models

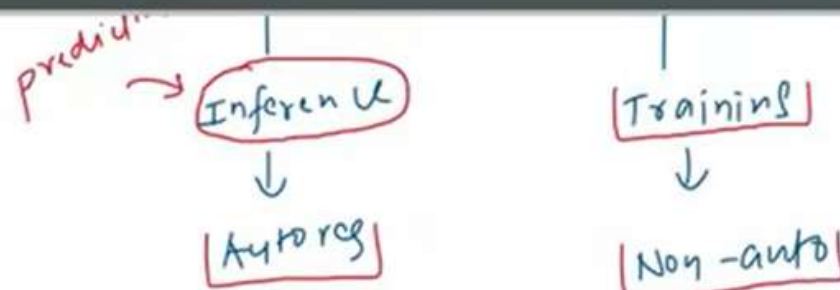
Autoregressive models

23 July 2024 17:39

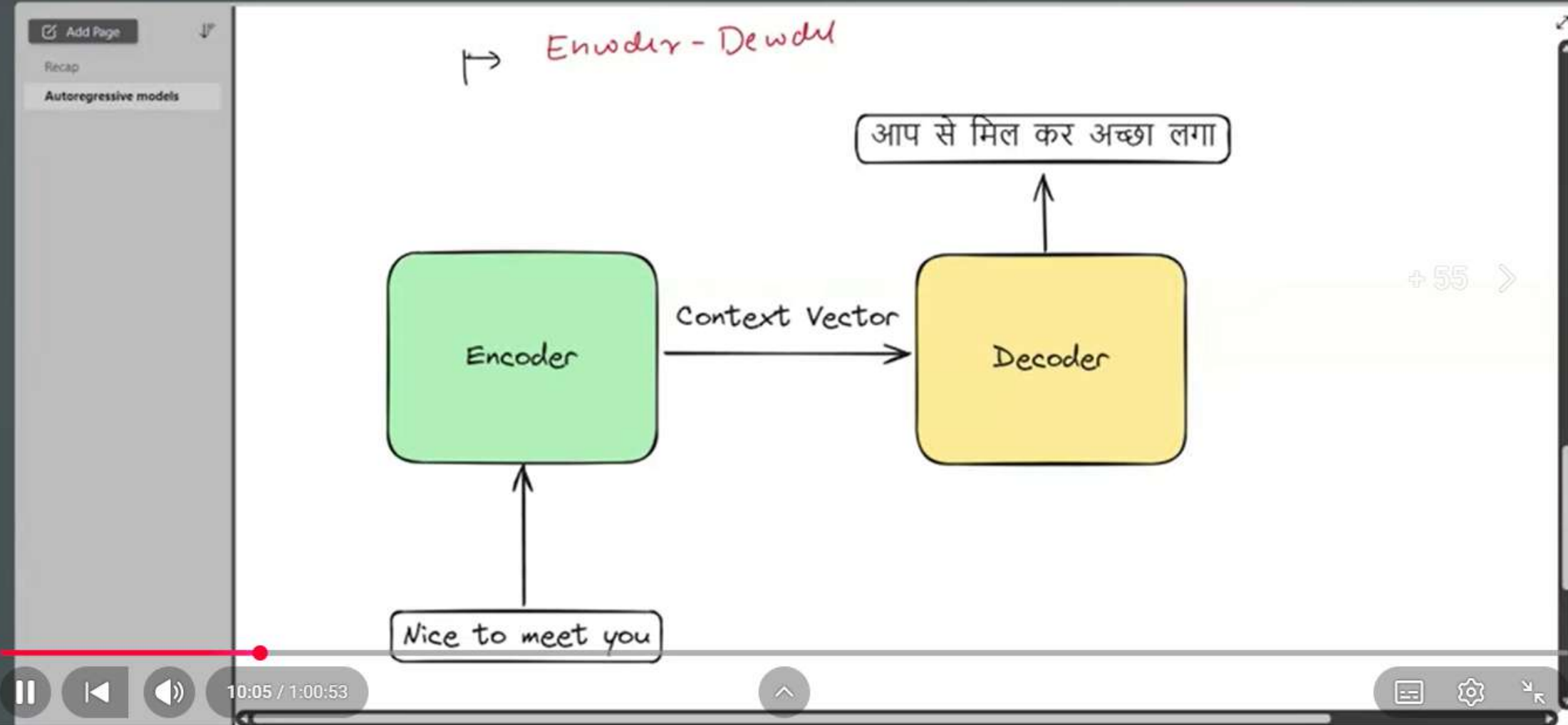
The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.



+ 40 >



In the context of deep learning, autoregressive models are a class of models that generate data points in a sequence by conditioning each new point on the previously generated points.



RecapAutoregr...

These Outputs are ignored

ht

ct

Nice to meet you

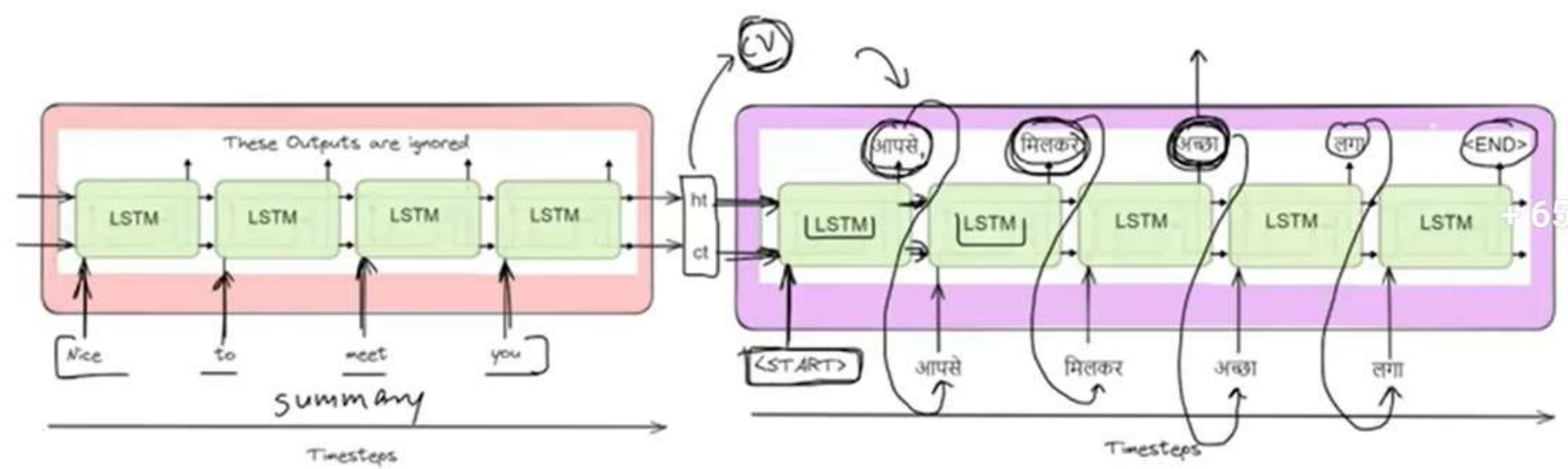
Timesteps

आपसे मिलकर अच्छा लगा <END>

<START> आपसे मिलकर अच्छा लगा

Timesteps

11:21 / 1:00:53



Transformer as an Autoregressive Model

23 July 2024 23:16

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time. ✓

Transformer as an Autoregressive Model

23 July 2024 23:16

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time. ✓

Recap

Autoregre...

Transfor...

TF Decoder

Inference - AR

Training - AR

eng

hindi

S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद

machine trans

TF

TF train

Inference

Query Sentence -> I am fine

Recap
Autoregre...
Transfor...

→ eng → hindi

S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद

machine trans → TF

TF train & inference

Inference

Query Sentence -> [I am fine]

मैं बर्दाश्त हूँ

मैं बर्दाश्त हूँ

मैं

बर्दाश्त

हूँ

<end>

ved

25:30 / 1:00:53

Recap

Autoregre...

Transfor...

→

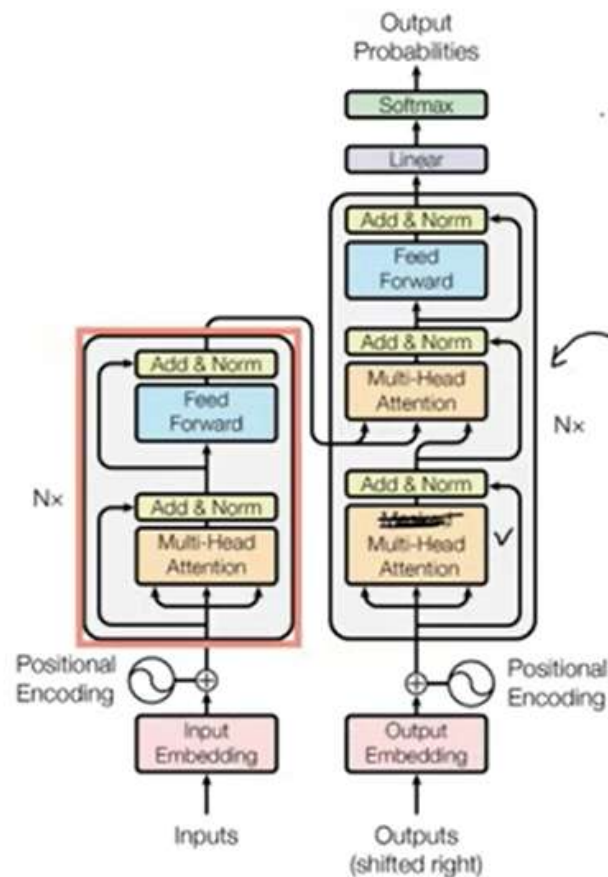
S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद

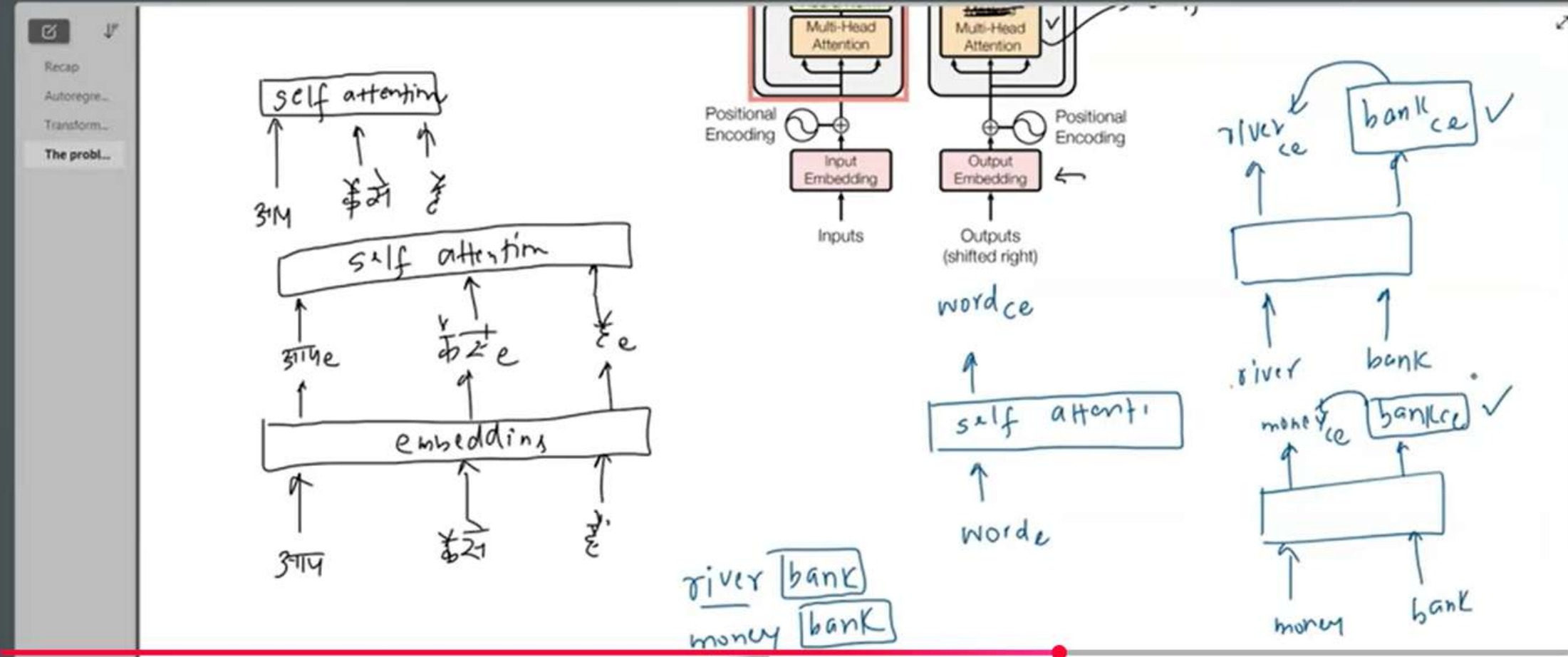
The diagram illustrates a sequence processing flow. It starts with a red block containing horizontal lines, with three upward arrows above it and three upward arrows below it. This is followed by a solid green block, then a light green block, and finally two more light green blocks. A red horizontal line spans the width of the diagram, with the text 'How are you?' written below it on the left side. At the bottom, there are navigation controls including a play/pause button, a back button, a volume icon, a progress bar showing '26:27 / 1:00:53', and a settings icon.

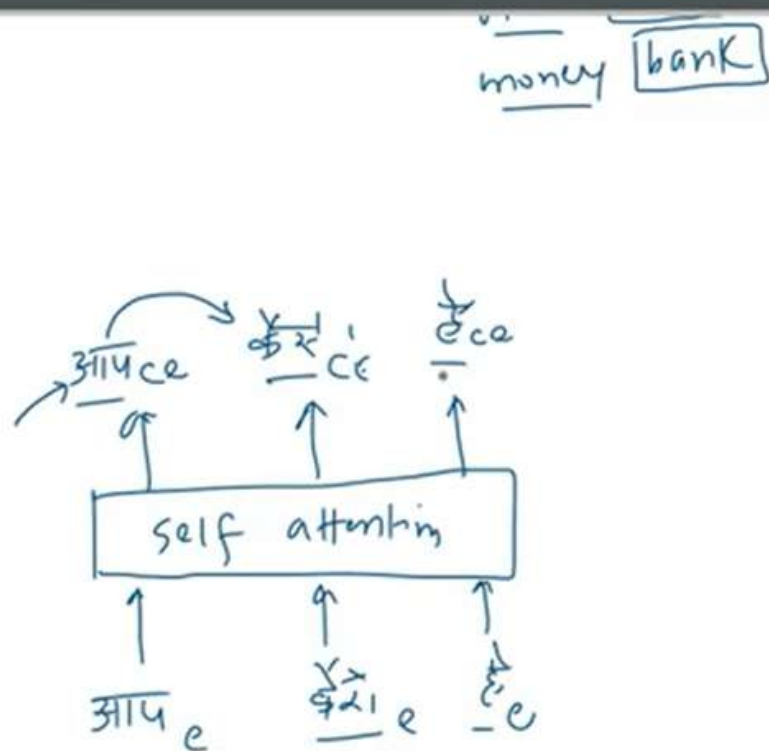
The problem in parallelizing

25 July 2024 22:55

S.No	English Sentence	Hindi Sentence
1	How are you?	आप कैसे हैं
2	Congratulations	बधाई हो
3	Thank you	धन्यवाद







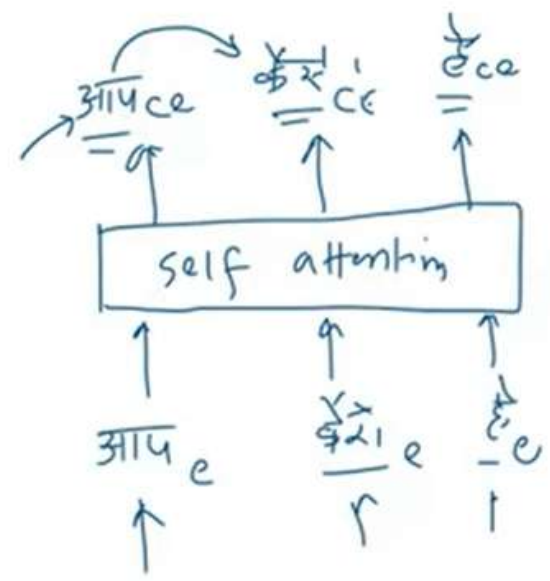
Recap
Autoregre...
Transform...
The probl...

money bank

money bank

$$\overline{A}u_{ce} = 0.8 \overline{A}u_e + 0.1 \overline{K}u_{ce} + 0.1 \overline{V}u_{ce}$$
$$\overline{K}u_{ce} = 0.15 \overline{A}u_e + 0.75 \overline{K}u_e + 0.1 \overline{V}u_e$$
$$\overline{V}u_{ce} = 0.1 \overline{A}u_e + 0.2 \overline{K}u_e + 0.7 \overline{V}u_e$$

Data leakage



cheating

आप क्या है

$$\overline{आप}_{ce} = 0.8 \overline{आप}_e + 0.1 \overrightarrow{क्या}_e + 0.1 \overleftarrow{है}_e$$

$$\overrightarrow{क्या}_{ce} = 0.15 \overline{आप}_e + 0.75 \overrightarrow{क्या}_e + 0.1 \overleftarrow{है}_e$$

$$\overleftarrow{है}_{ce} = 0.1 \overline{आप}_e + 0.2 \overrightarrow{क्या}_e + 0.7 \overleftarrow{है}_e$$

current token value
↓
future token value

- Recap
- Autoregre...
- Transform...
- The probl...

↑
cheating

current token
↓
future token value

<u>AR</u>	<u>NAR</u>
→ No Data Leakage	→ fast
→ slow	→ Data leakage

Duvidha

↑

cheating

current token

future token value

AR

→ No Data Leakage

→ slow

NAR

→ fast

→ Data leakage

Duvidha

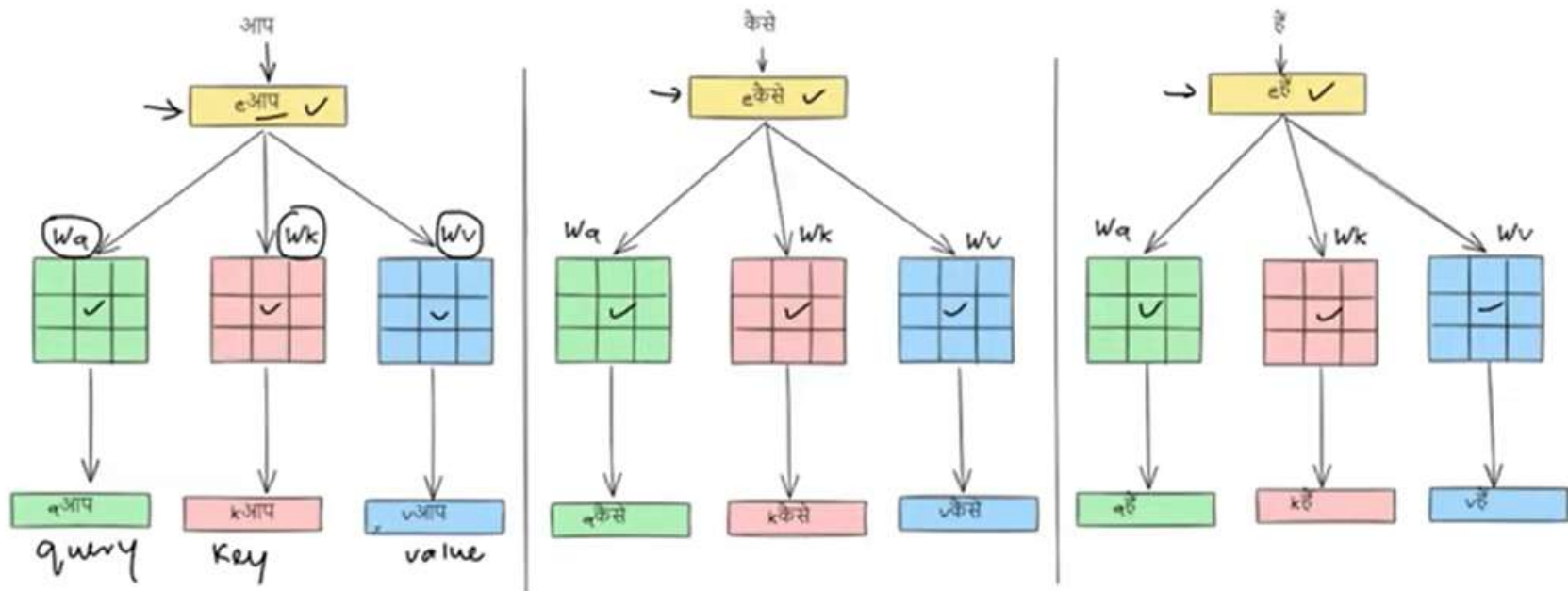
yes

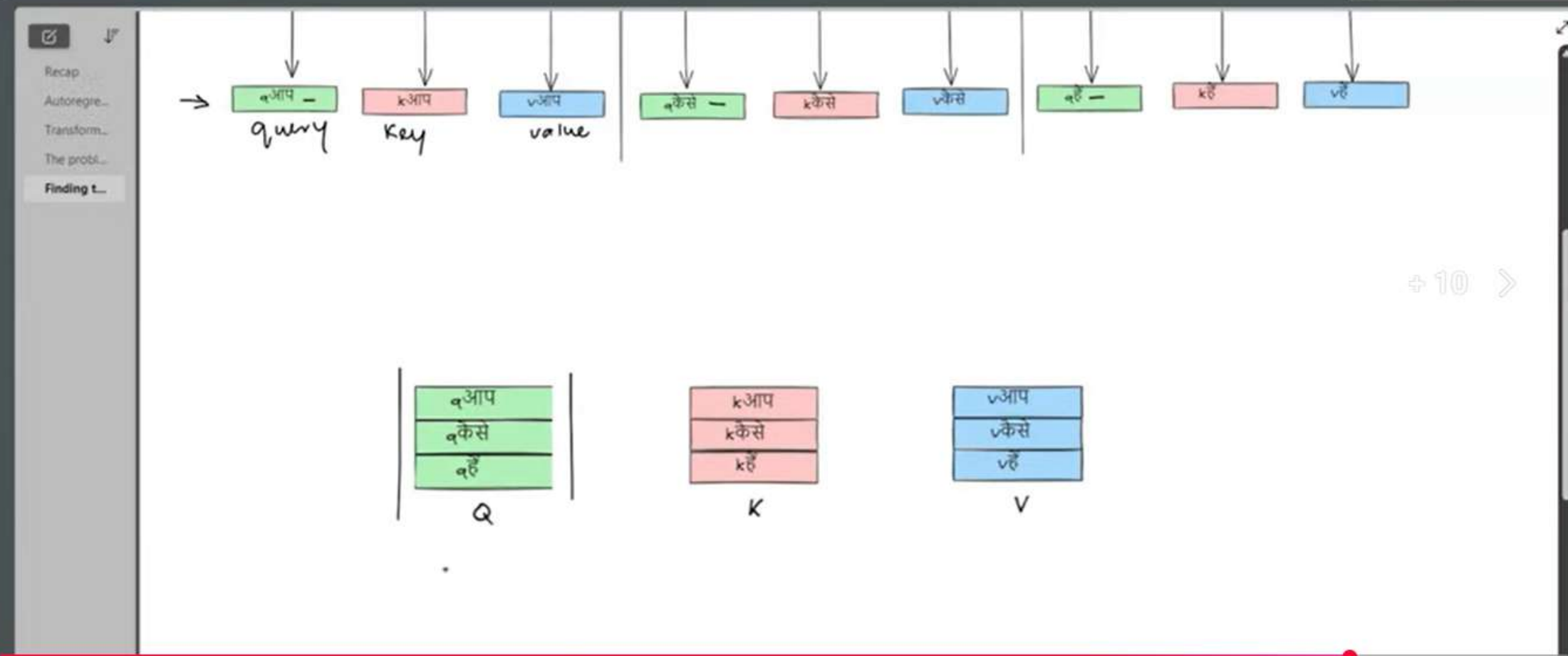
self attention

how

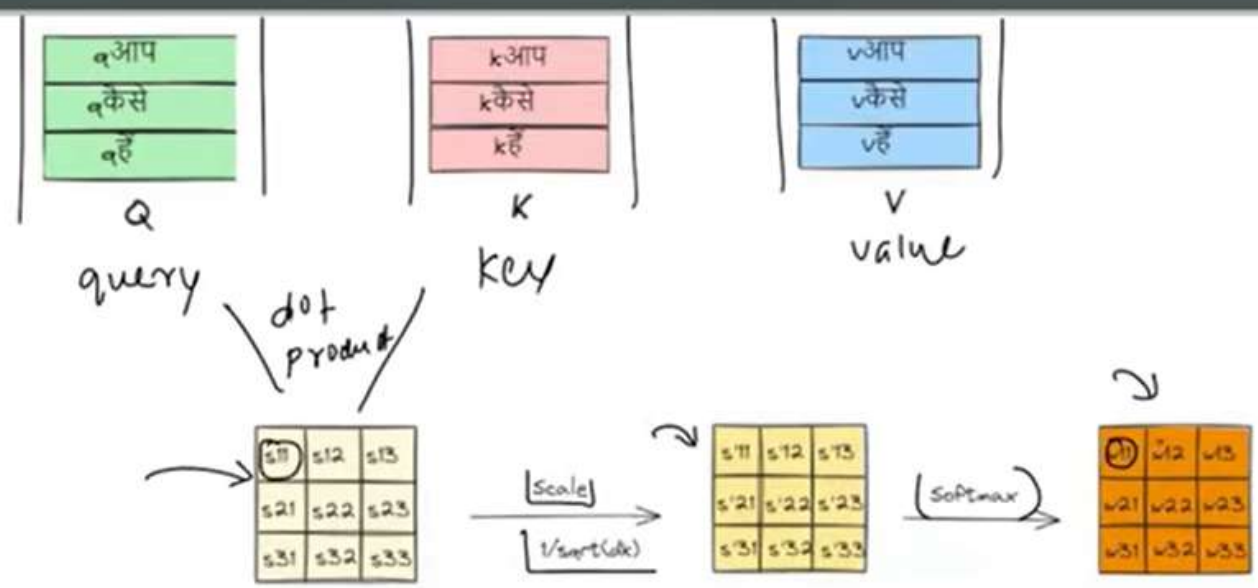
Finding the answer

26 July 2024 00:21





- Recap
- Autoregre...
- Transform...
- The probl...
- Finding t...



$$\begin{aligned} \text{आप}ce &= w_{11} * \text{vआप} + w_{12} * \text{vकेसे} + w_{13} * \text{vहै} \\ \text{केसे}ce &= w_{21} * \text{vआप} + w_{22} * \text{vकेसे} + w_{23} * \text{vहै} \\ \text{है}ce &= w_{31} * \text{vआप} + w_{32} * \text{vकेसे} + w_{33} * \text{vहै} \end{aligned}$$

Diagram illustrating the calculation of the output for the first row of the attention matrix (e.g., for the word "आप") using the dot product of the input vector and the weight matrix.

Input vector (row 1): s_{11}, s_{12}, s_{13}

Weight matrix (columns): w_{11}, w_{12}, w_{13}

Calculation for the first row:

$$e_{11} = w_{11} * s_{11} + w_{12} * s_{12} + w_{13} * s_{13}$$
$$e_{21} = w_{21} * s_{11} + w_{22} * s_{12} + w_{23} * s_{13}$$
$$e_{31} = w_{31} * s_{11} + w_{32} * s_{12} + w_{33} * s_{13}$$

The resulting vector is then scaled by $1/\sqrt{d_k}$ and passed through a Softmax function to produce the attention weights.

Diagram illustrating the calculation of the output for the first row of the attention matrix (e.g., for the word "आप") using the dot product of the input vector and the weight matrix.

Input vector (row 1): s_{11}, s_{12}, s_{13}

Weight matrix (columns): w_{11}, w_{12}, w_{13}

Calculation for the first row:

$$e_{11} = w_{11} * s_{11} + w_{12} * s_{12} + w_{13} * s_{13}$$
$$e_{21} = w_{21} * s_{11} + w_{22} * s_{12} + w_{23} * s_{13}$$
$$e_{31} = w_{31} * s_{11} + w_{32} * s_{12} + w_{33} * s_{13}$$

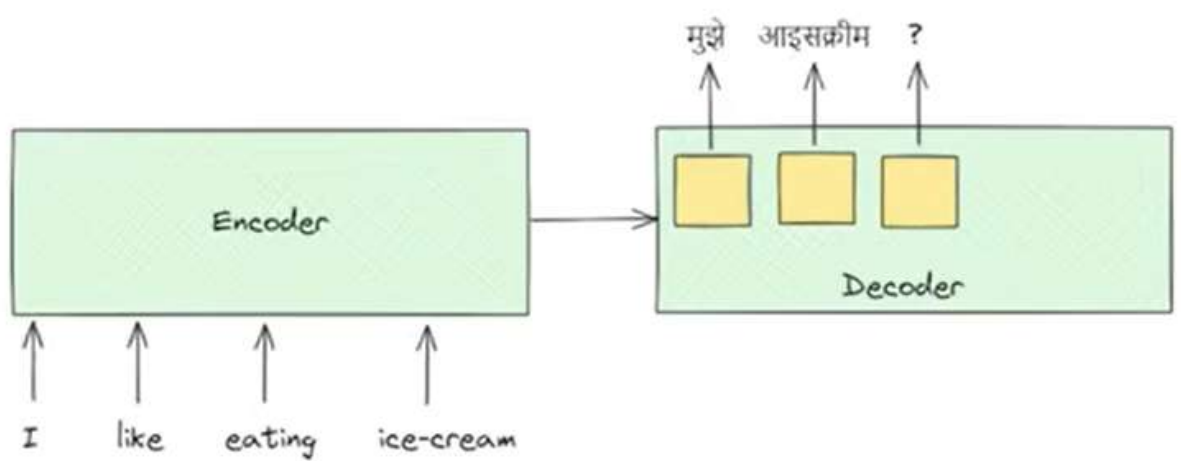
The resulting vector is then scaled by $1/\sqrt{d_k}$ and passed through a Softmax function to produce the attention weights.

$w_{12} = w_{13} = w_{23} = 0$

What is Cross Attention

12 August 2024 17:53

Cross-attention is a mechanism used in transformer architectures, particularly in tasks involving sequence-to-sequence data like translation or summarization. It allows a model to focus on different parts of an input sequence when generating an output sequence.



मुझे
I
like
eating
ice-cream

Plan of Ac...

What is C...

Self-Atten...

Self-Atten...

Self-Atten...

Cross Atte...

Use-cases

es, particularly in tasks
rization. It allows a model to
an output sequence.

translation

$t=3$

→ 1) what generated
fill now

2) input sentence

self attention

मुझे आइसक्रीम ? खाना

[Decoder]

I

like

eating

ice-cream

	मुझे	आइसक्रीम	खाना	पसंद	है
I	●	●	●	●	●
like	●	●	●	●	●
eating	●	●	●	●	●
ice-cream	●	●	●	●	●

9:16 / 34:06



Plan of Ac...

What is C...

Self-Atten...

Self-Atten...

Self-Atten...

Cross Atte...

Use-cases

< - 10

Cross Attention is conceptually very similar to Self-Attention

Self-Attention Vs Cross Attention

- 1. The input ✓
- 2. The processing ✓
- 3. The output ✓

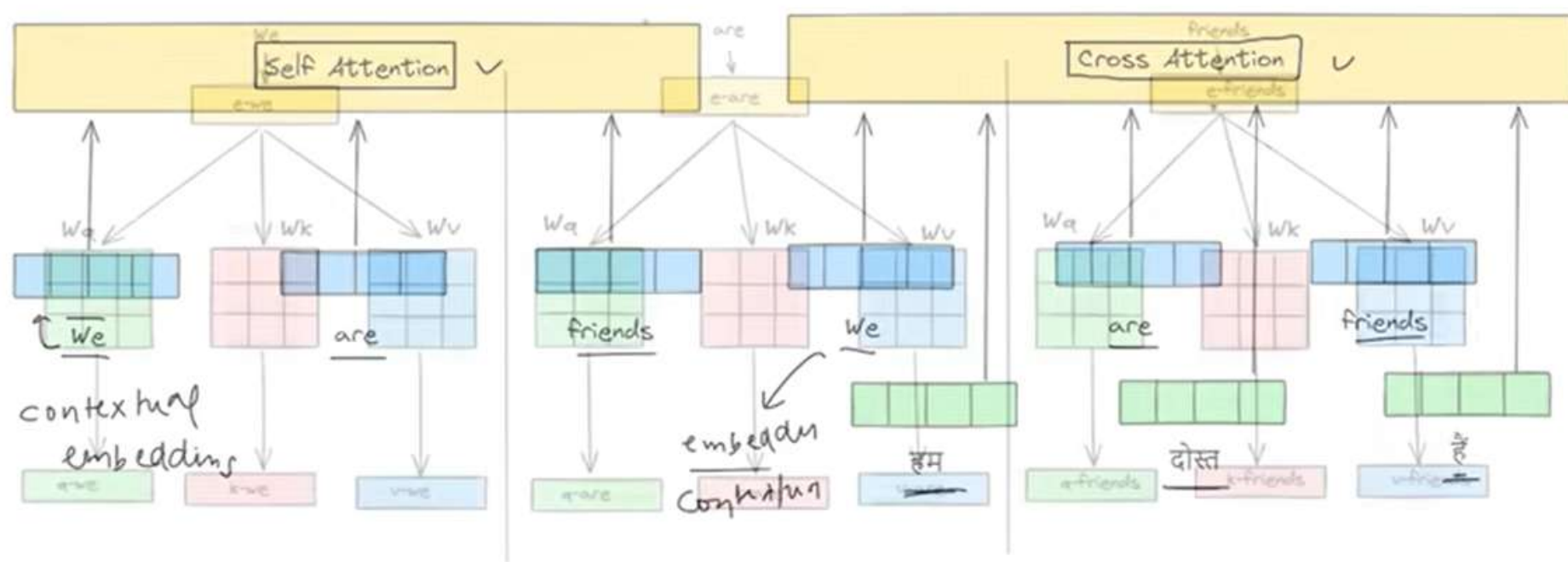


12:31 / 34:06



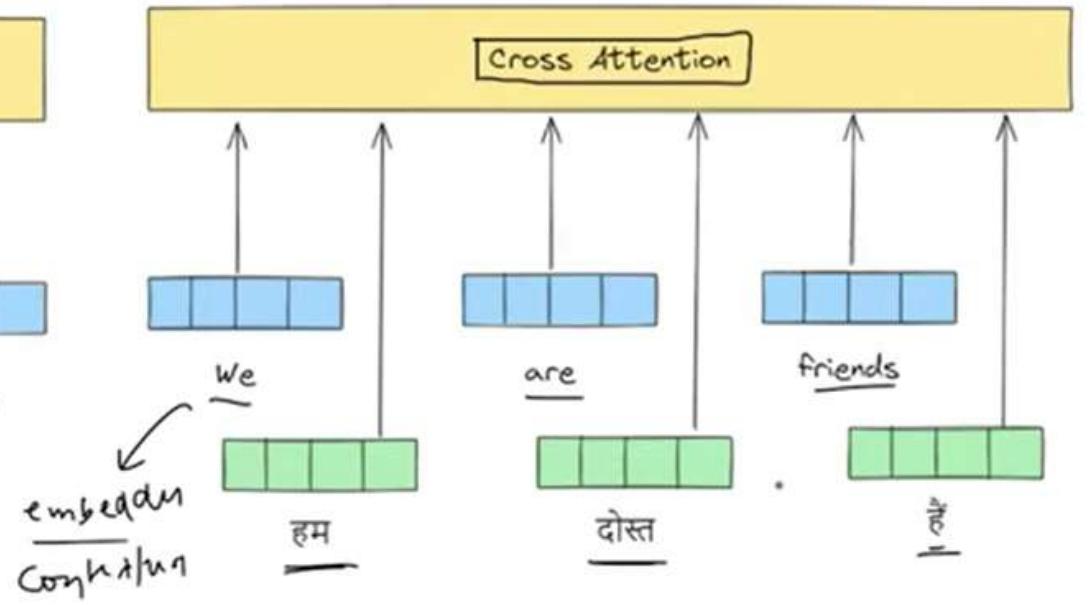
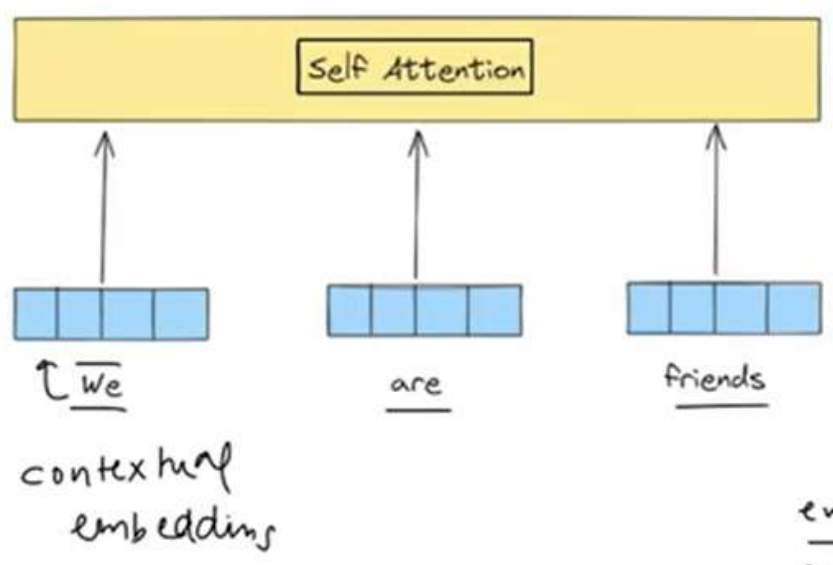
Self-Attention Vs Cross Attention (Input Encoding)

13 August 2024 08:22



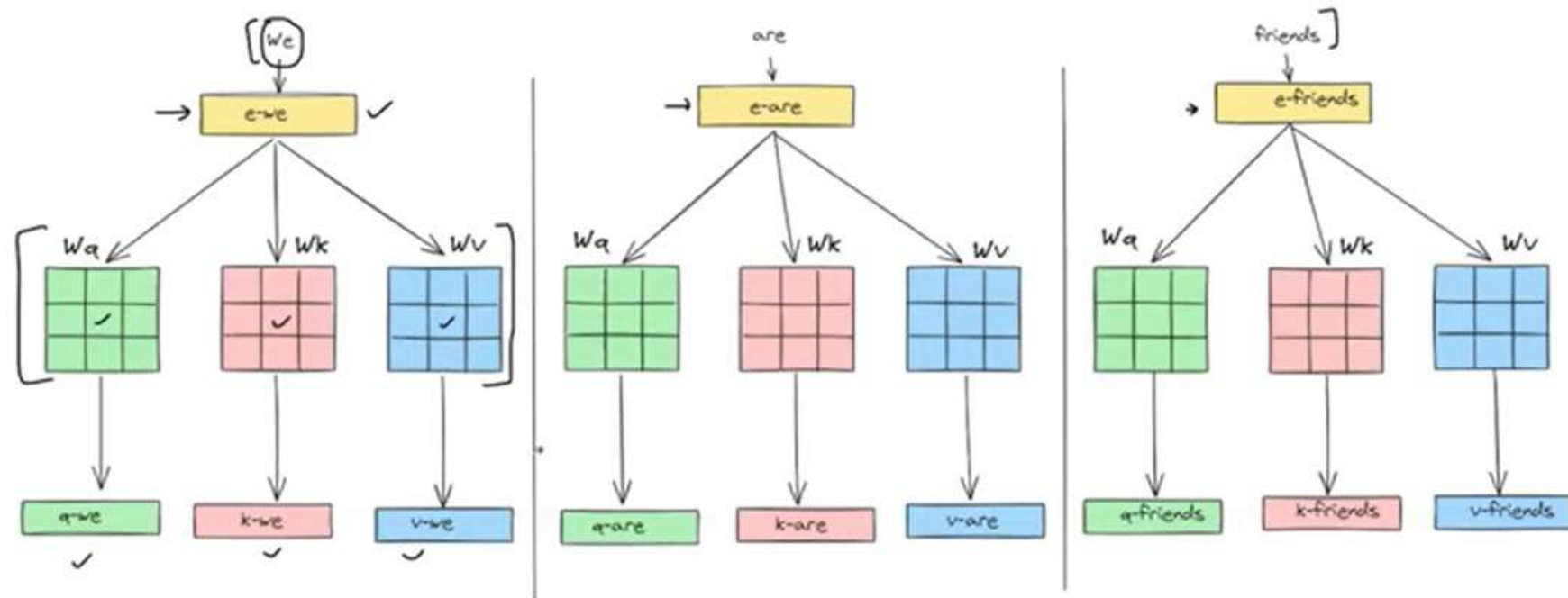
Self-Attention Vs Cross Attention (Input)

13 August 2024 08:22



Self-Attention Vs Cross Attention (Processing)

12 August 2024 17:54



16:31 / 34:06

q-we s12

q-we s13

q-are s21

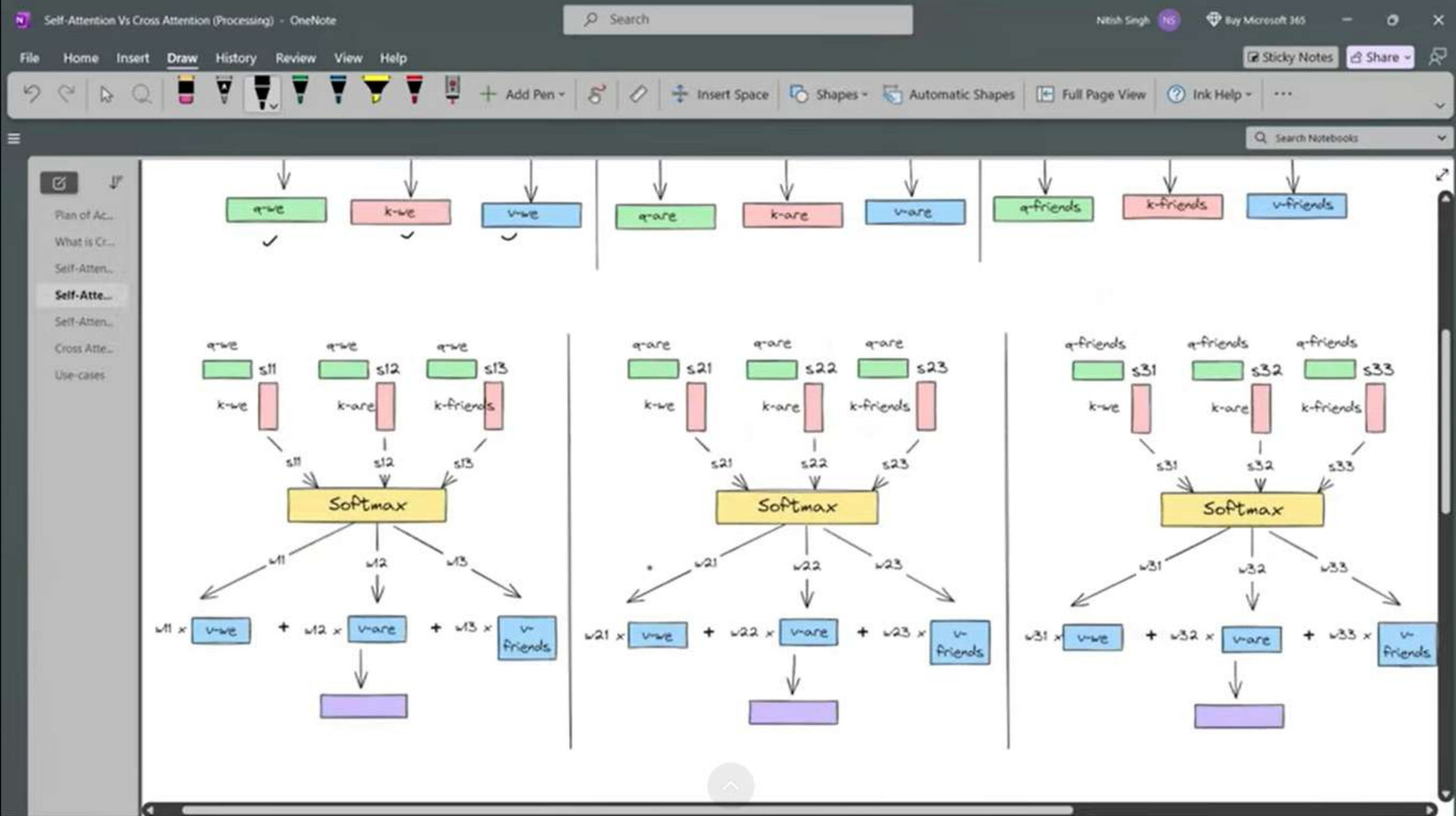
q-are s22

q-are s23

q-friends <21

q-friends <22

q-friends <23



Plan of Ac...
What is Cr...
Self-Atten...
Self-Atte...
Self-Atten...
Cross Atte...
Use-cases

Diagram 1: q_{we} , k_{we} , s_{11} , s_{12} , s_{13} → Softmax → $w_{11} \times v_{we}$, $w_{12} \times v_{are}$, $w_{13} \times v_{friends}$

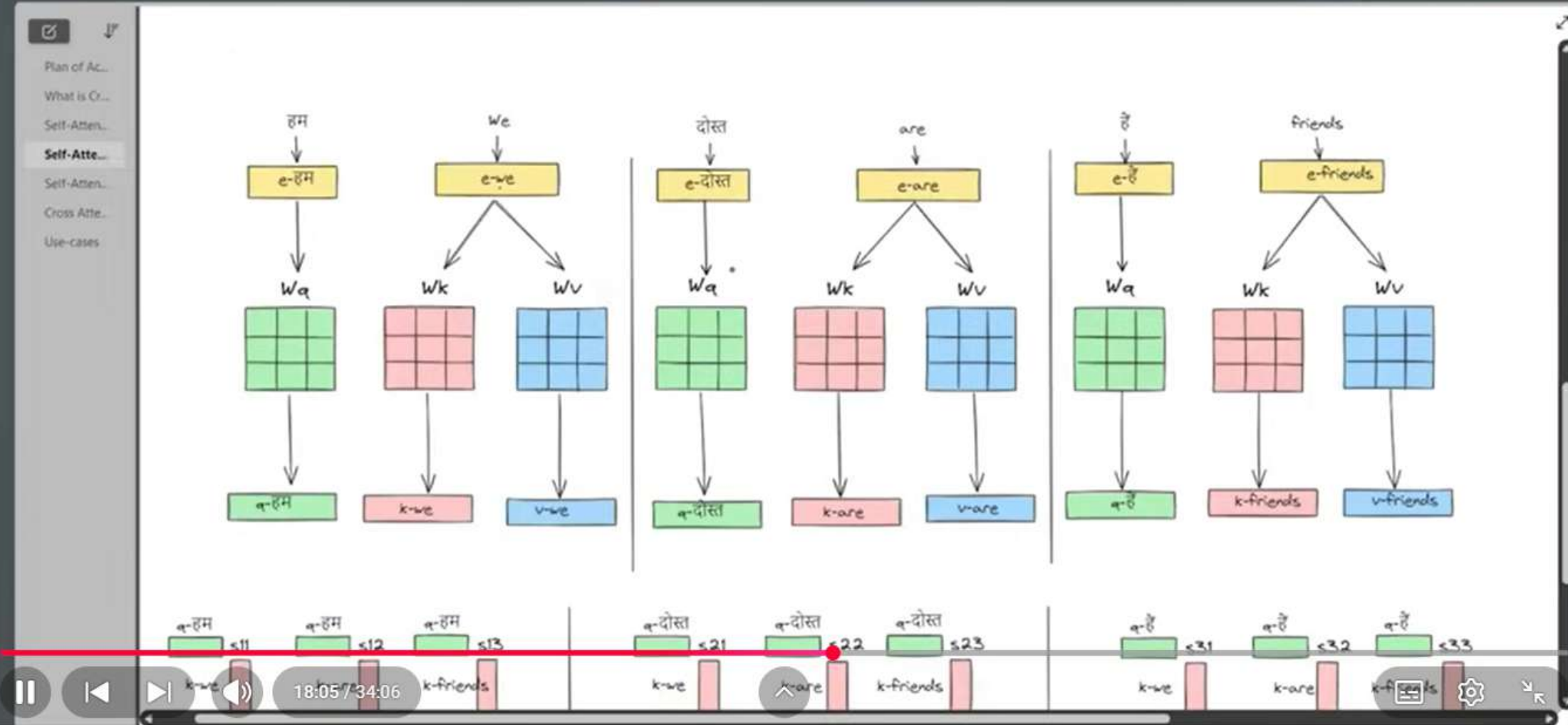
Diagram 2: q_{are} , k_{are} , s_{21} , s_{22} , s_{23} → Softmax → $w_{21} \times v_{we}$, $w_{22} \times v_{are}$, $w_{23} \times v_{friends}$

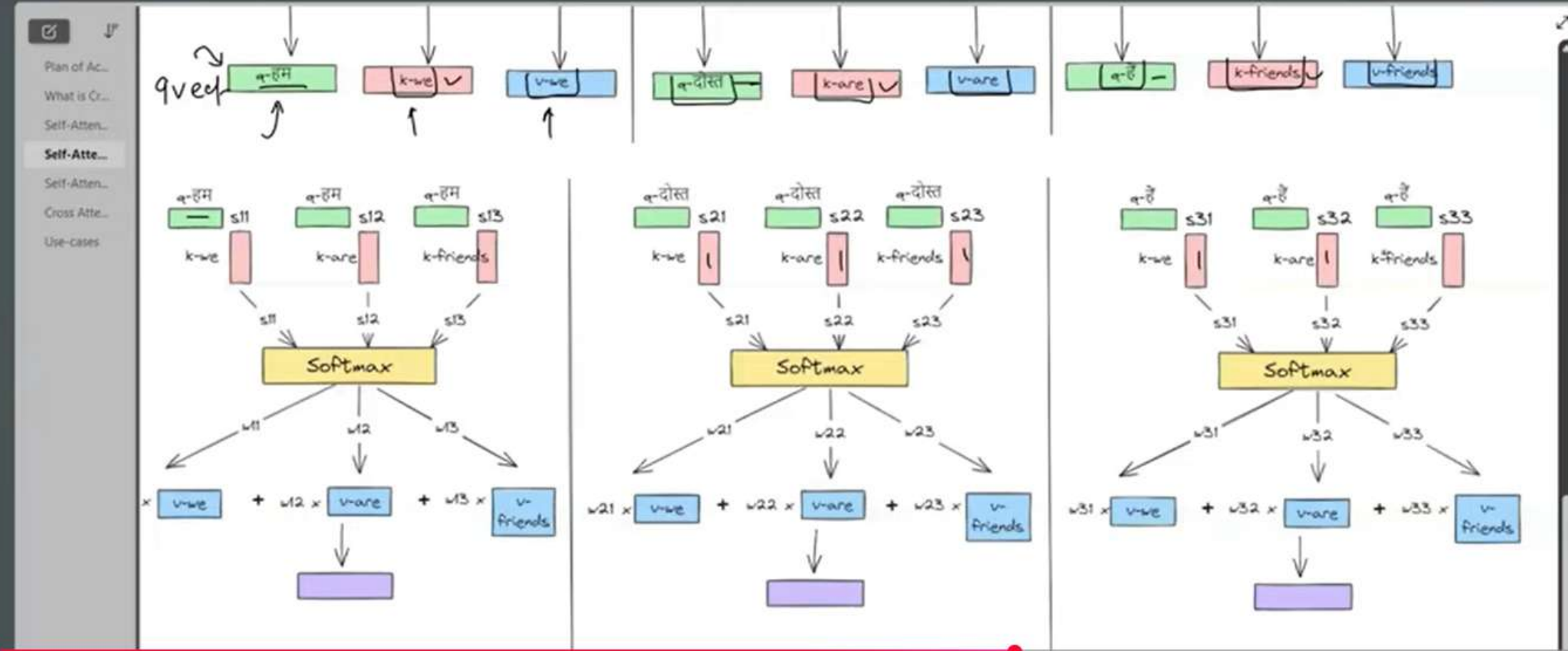
Diagram 3: $q_{friends}$, $k_{friends}$, s_{31} , s_{32} , s_{33} → Softmax → $w_{31} \times v_{we}$, $w_{32} \times v_{are}$, $w_{33} \times v_{friends}$

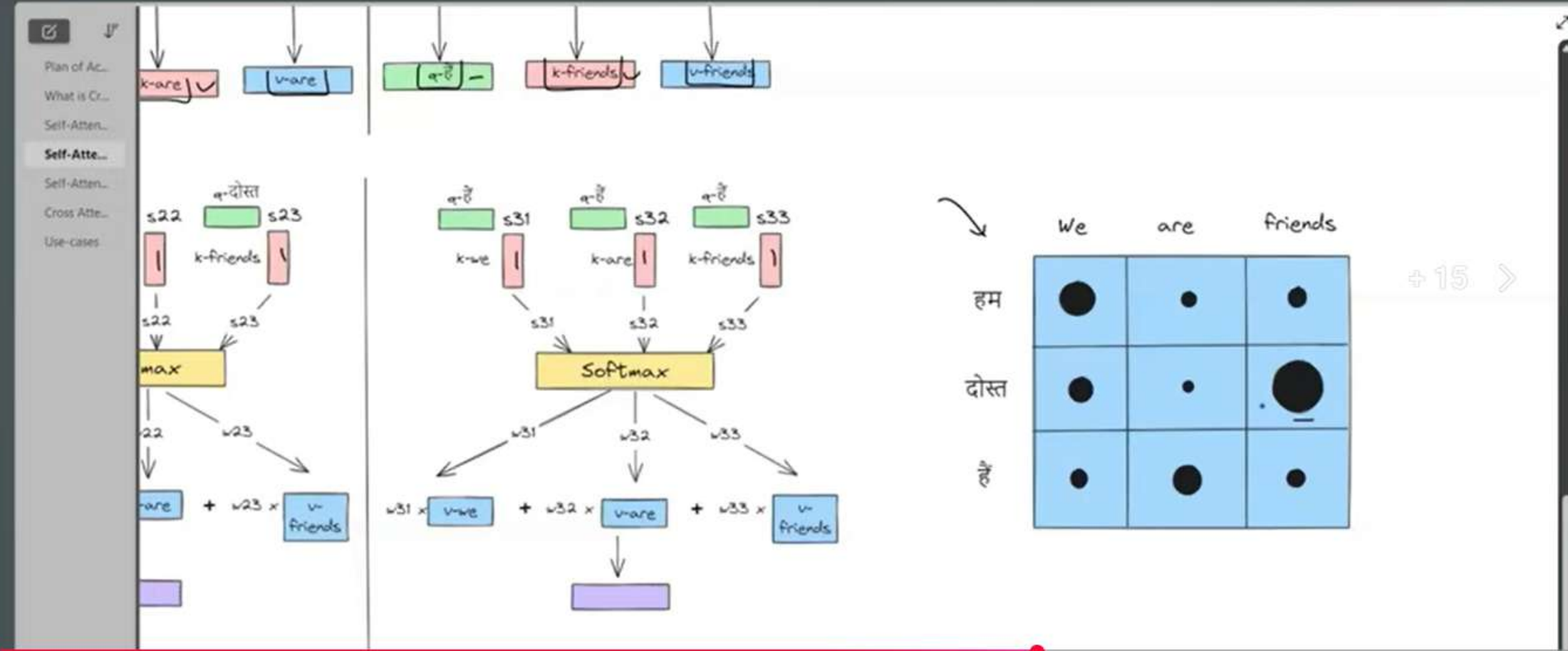
Timeline: हम (e-ham), we (e-we), दोस्त (e-dost), are (e-are), is (e-is), friends (e-friends)

17:43 / 34:06

Pause, Play, Previous, Next, Full Screen, Settings, Close

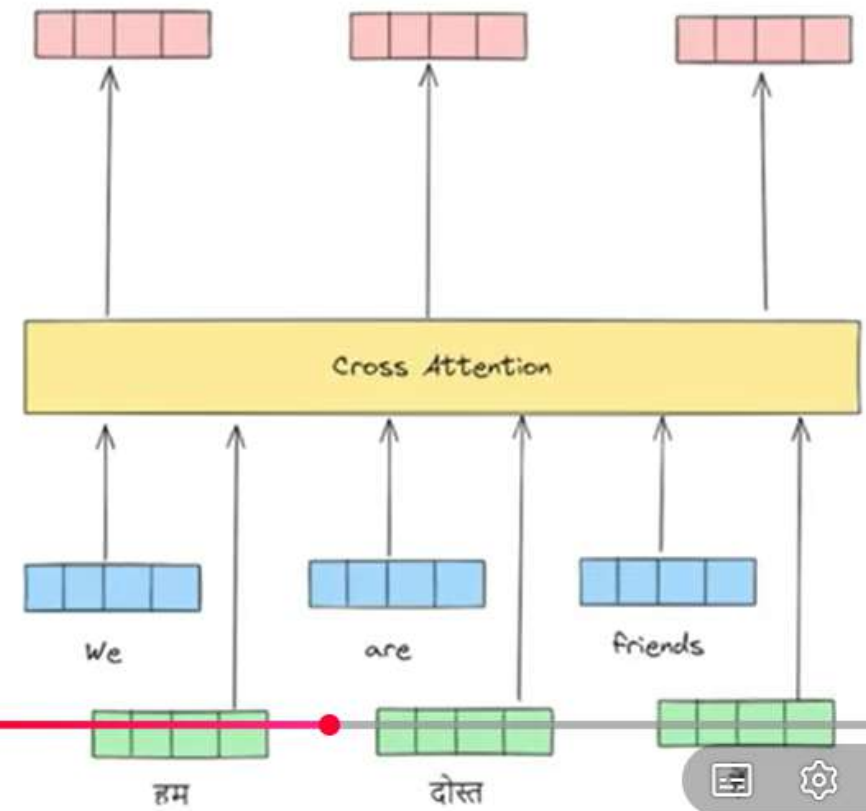
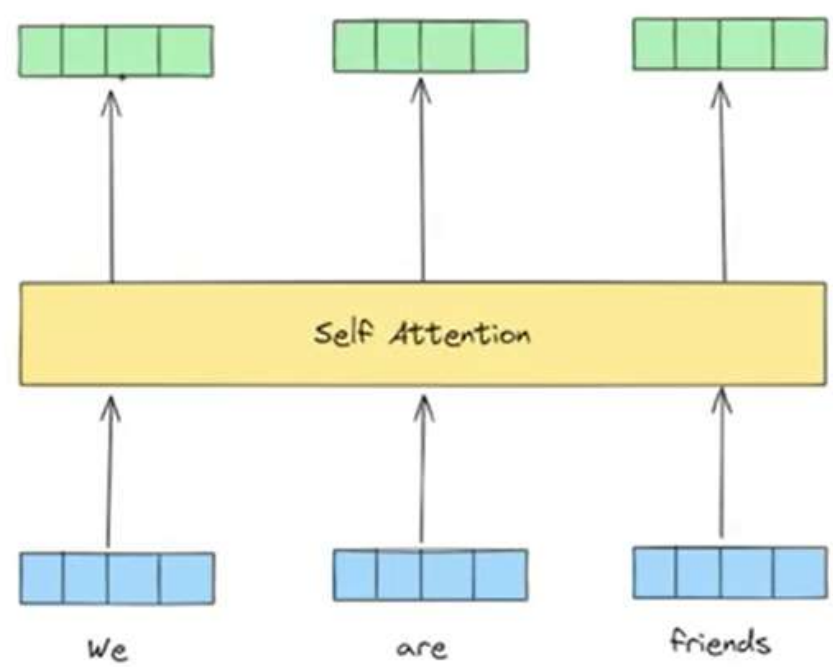






Self-Attention Vs Cross Attention [Output]

12 August 2024 18:05



Wearefriends

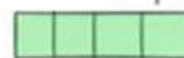
We

are

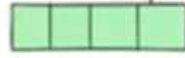
friends



हम



दोस्त

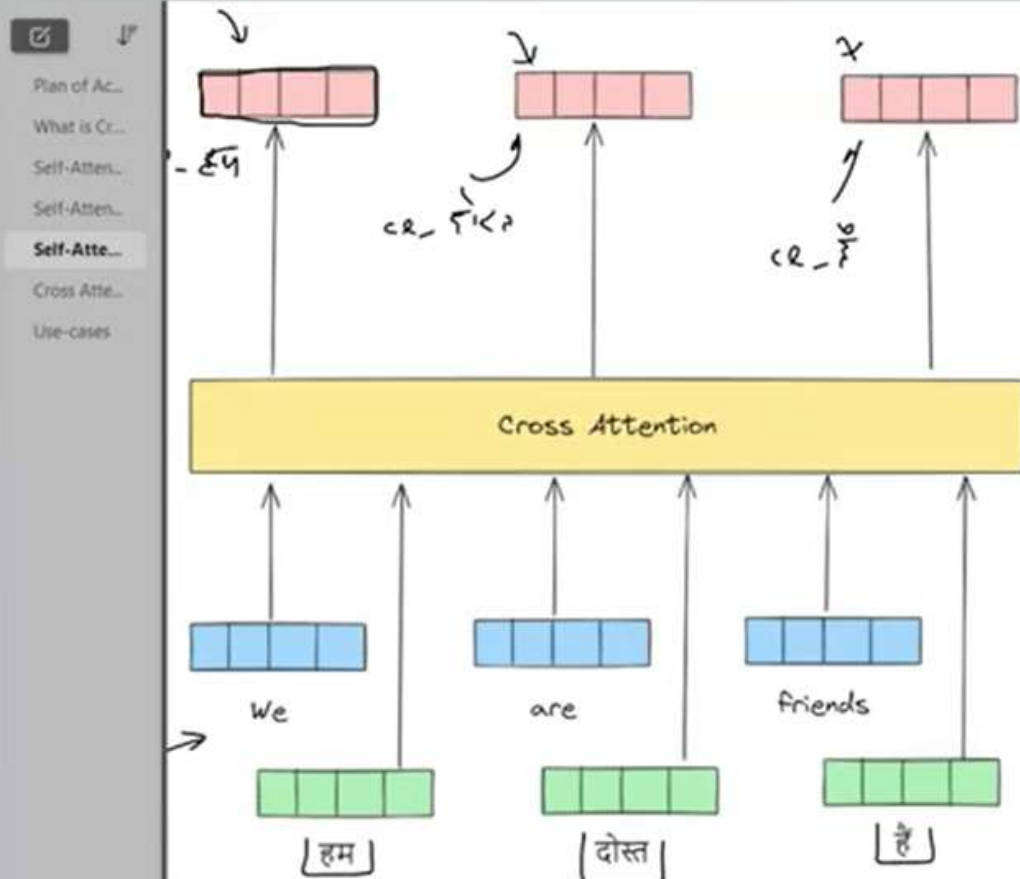


है

$$ce_we = 0.8 \times e_we + 0.1 \times e_are + 0.1 \times e_friends$$

$$ce_are = 0.15 \times e_we + 0.75 \times e_are + 0.1 \times e_friends$$

$$ce_friends = 0.2 \times e_we + 0.1 \times e_are + 0.7 \times e_friends$$

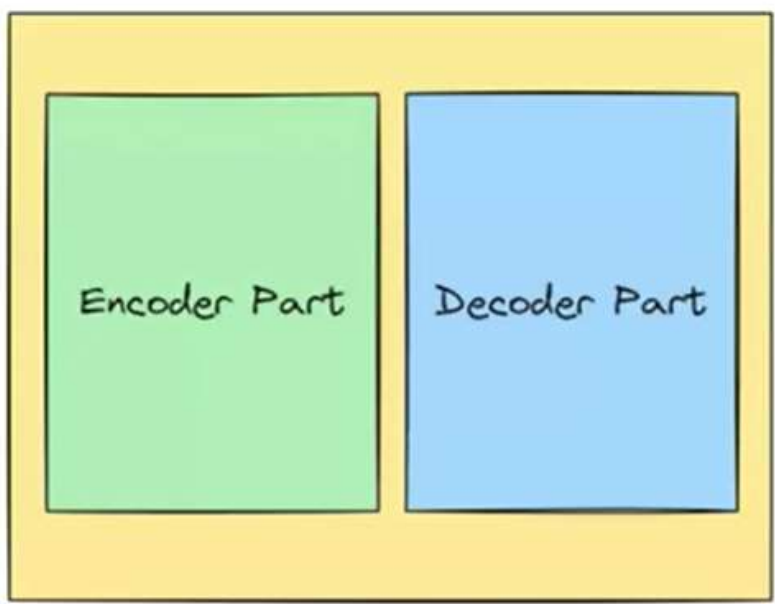
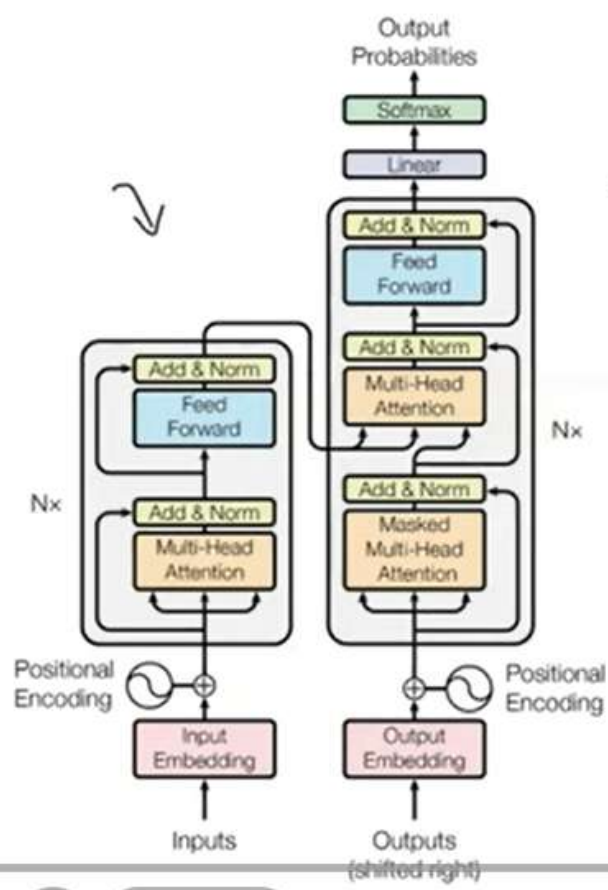


$$\underline{ce_हम} = 0.5 \times e_we + 0.3 \times e_are + 0.2 \times e_friends$$

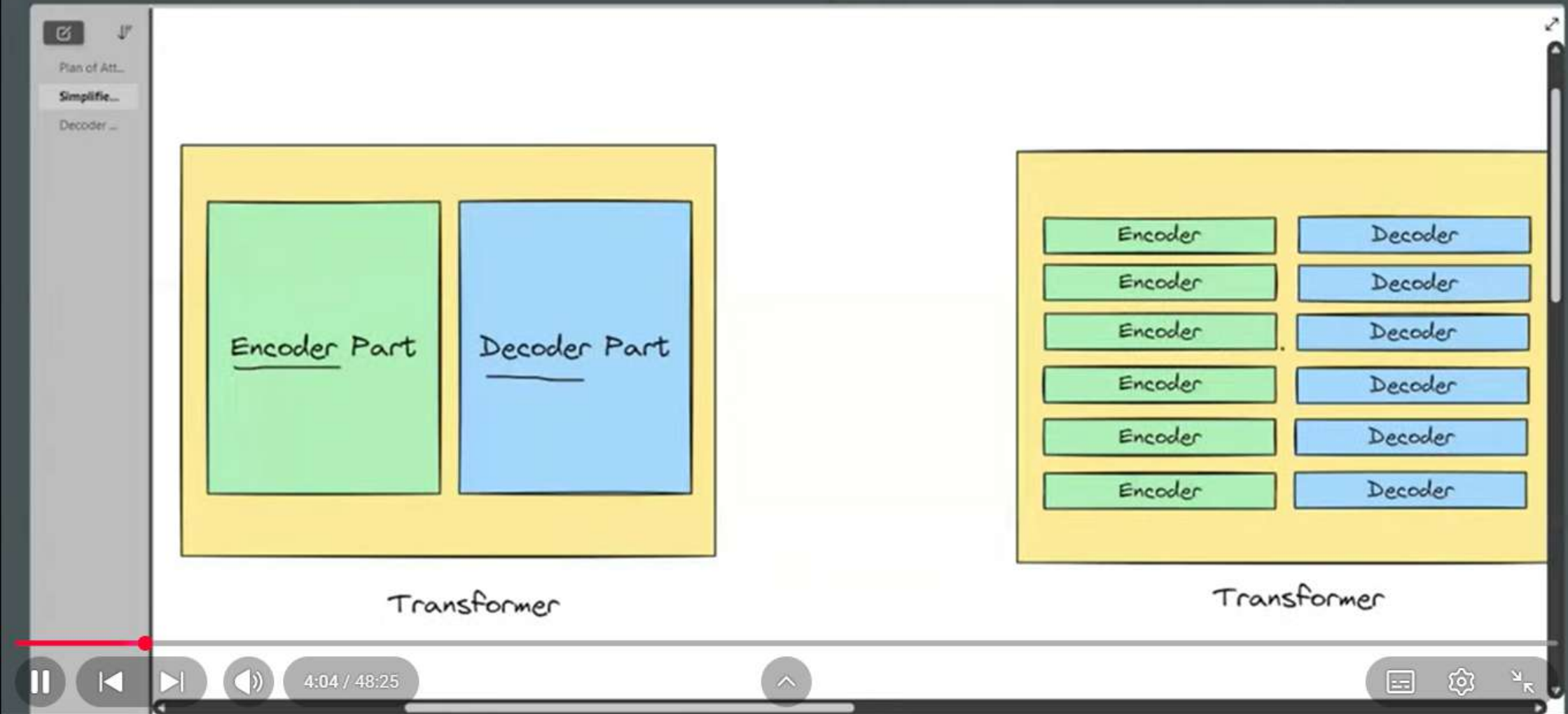
$$\underline{ce_दोस्त} = 0.2 \times e_we + 0.2 \times e_are + 0.6 \times e_friends$$

$$\underline{ce_हैं} = 0.3 \times e_we + 0.4 \times e_are + 0.3 \times e_friends$$

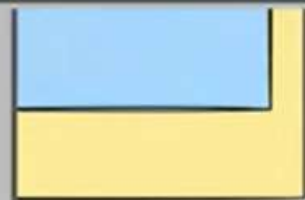
+75 >



Transformer



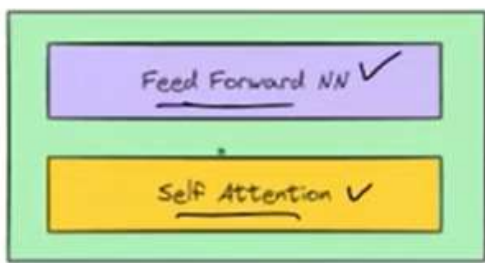
Plan of Att...
Simplifie...
Decoder ...

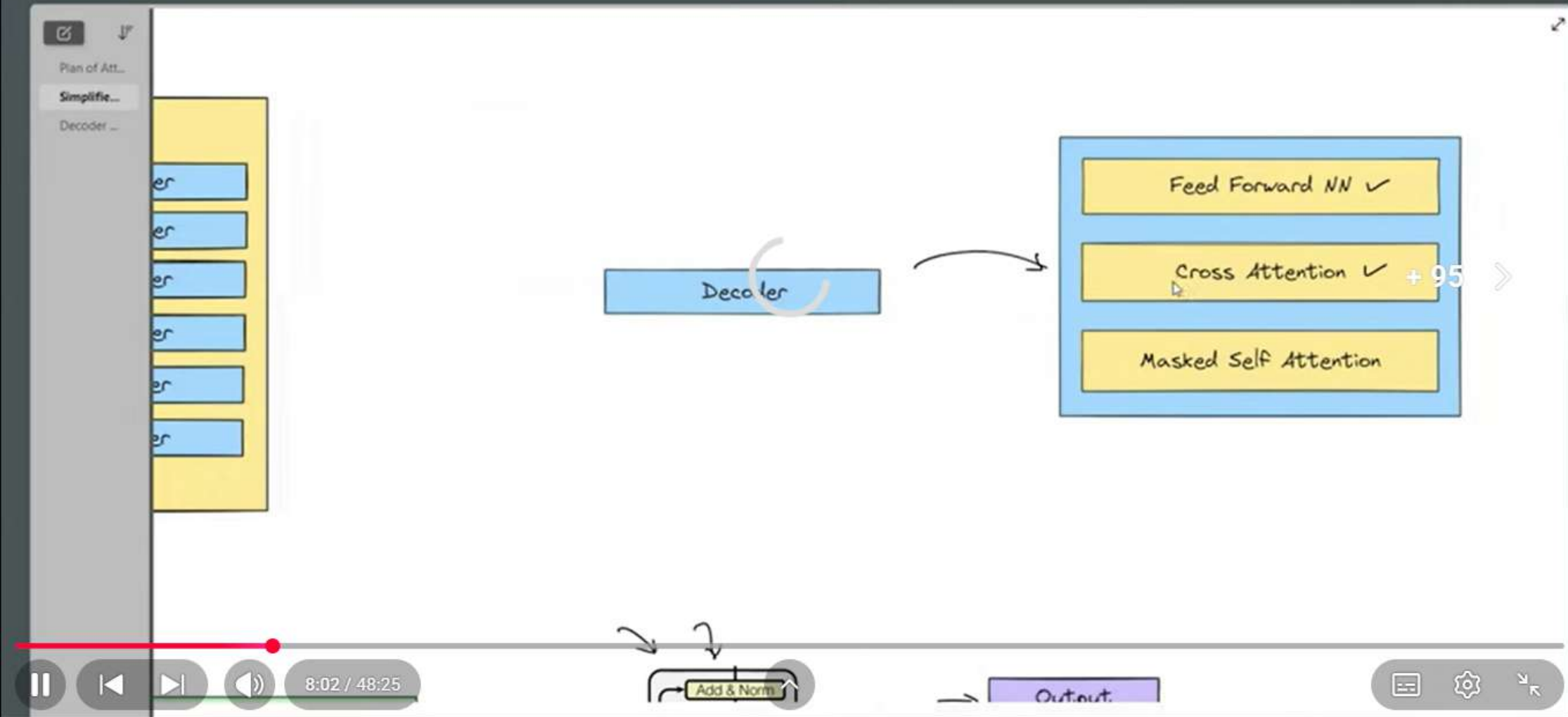


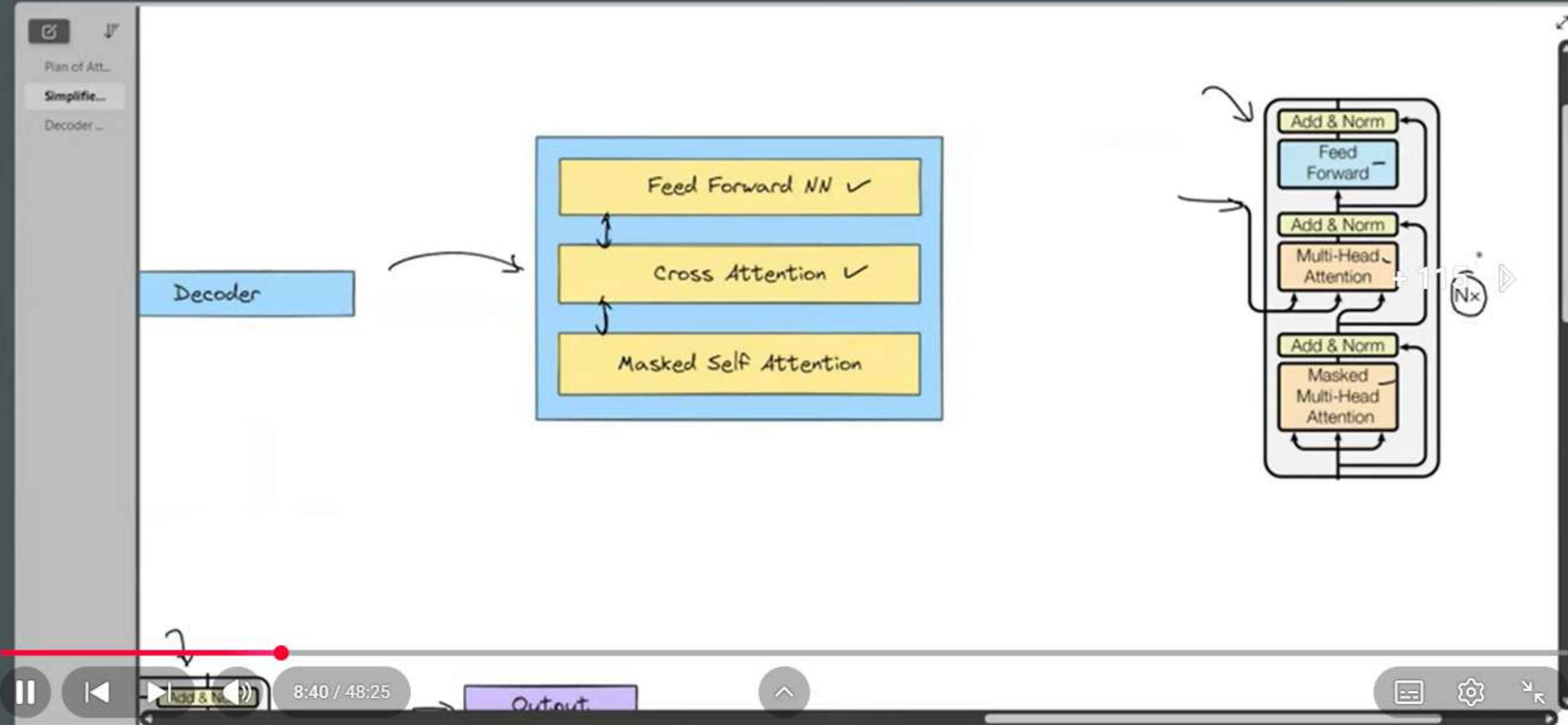
Transformer

+30 >

Encoder







Machine Translation Task

English Sentence - We are friends

Hindi Sentence - Hum dost hai

Machine Translation Task

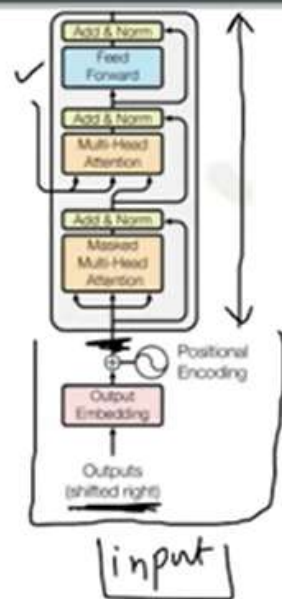
English Sentence - We are friends

→ Training

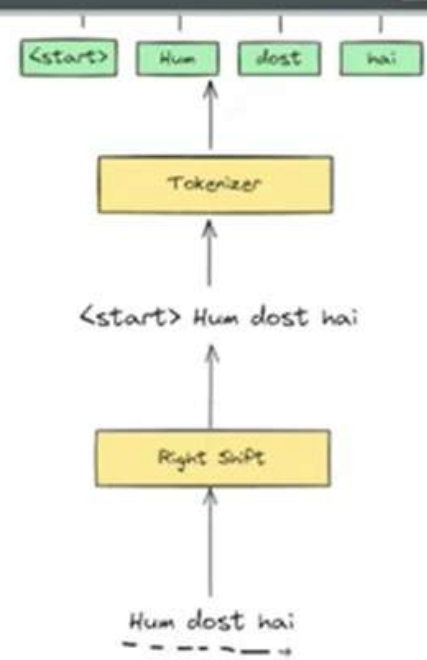
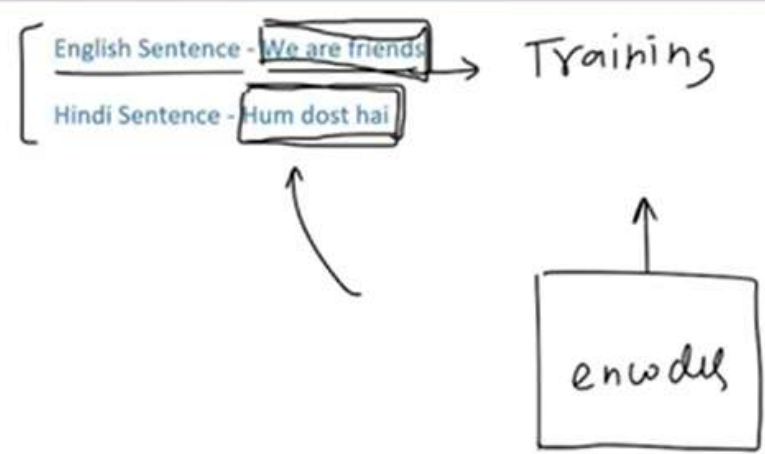
Hindi Sentence - Hum dost hai

12:06 / 48:25

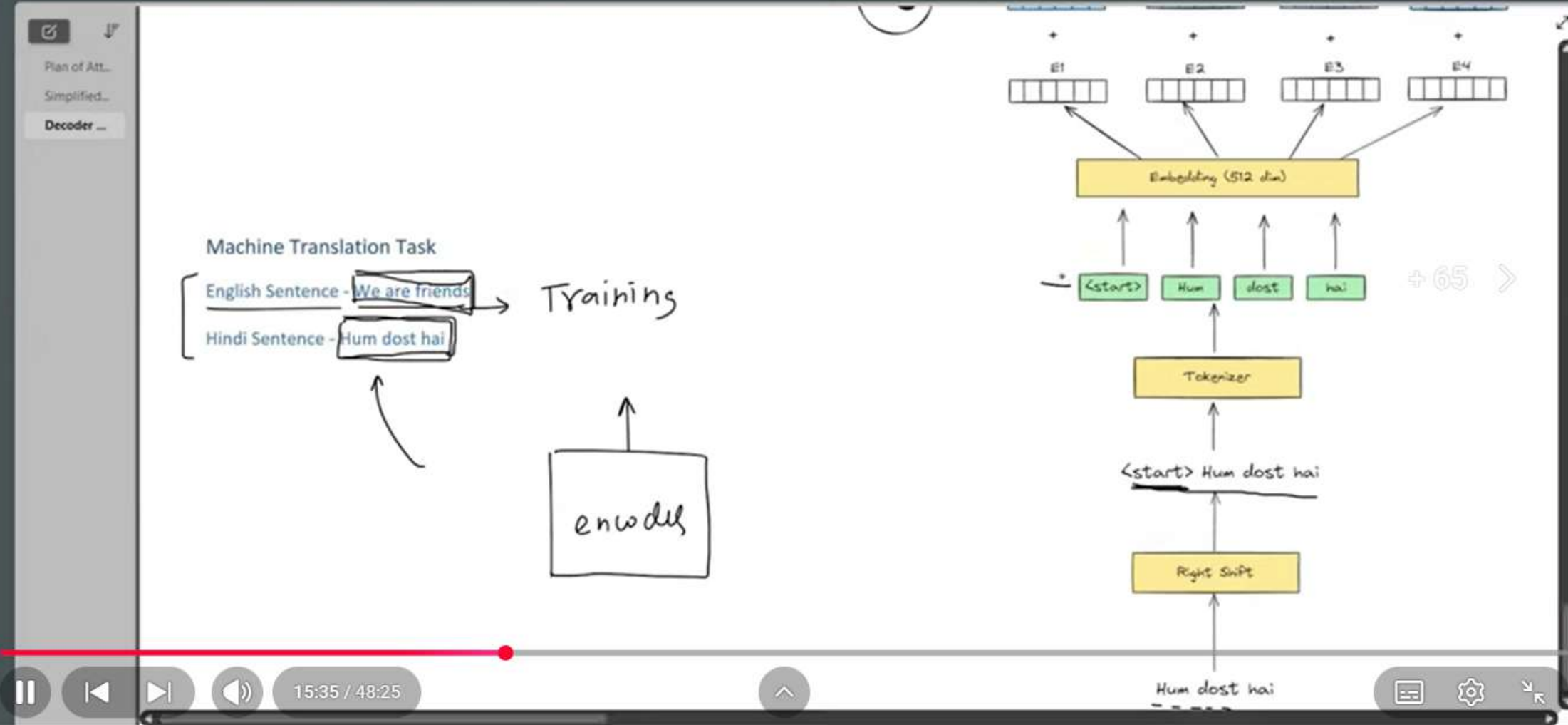


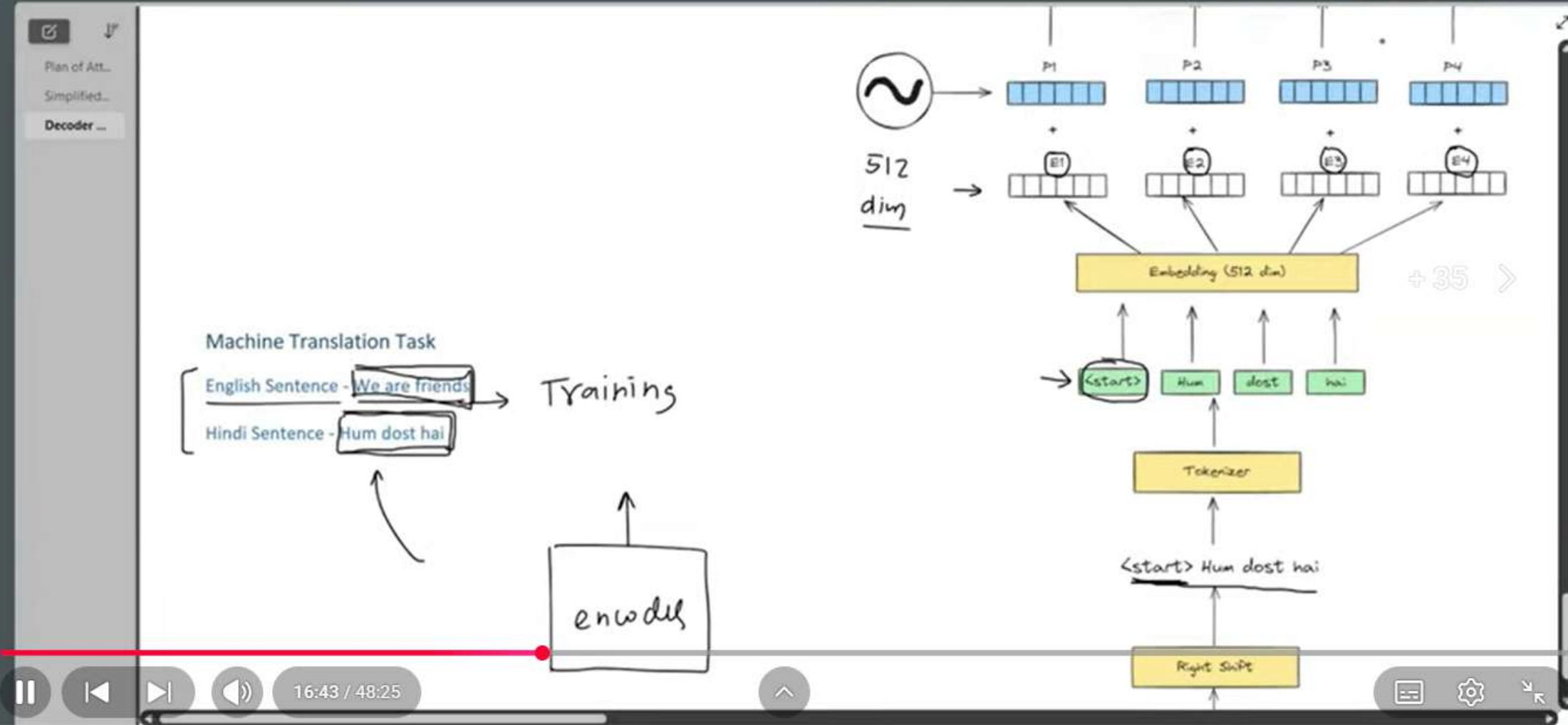


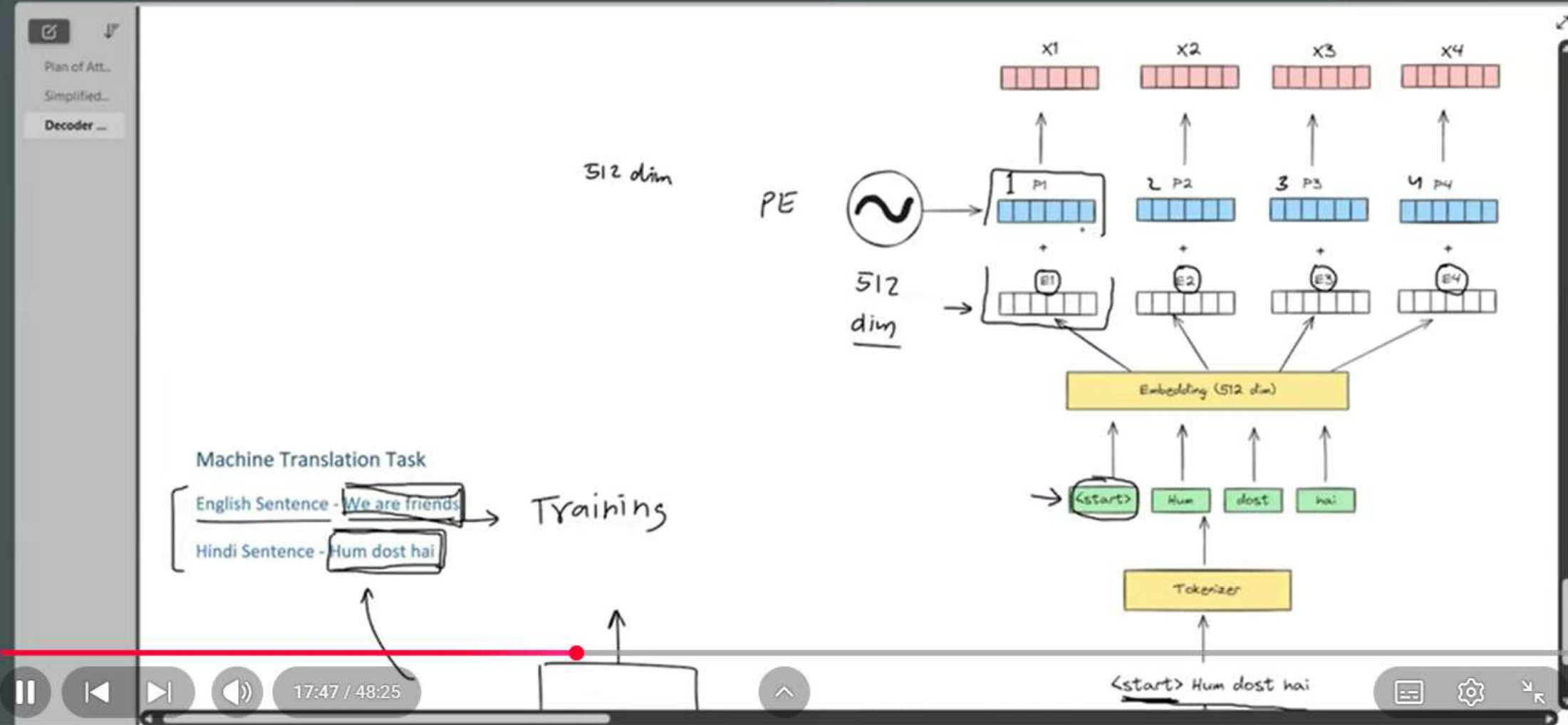
1. Shifting
2. Tokenization
3. Embedding
4. Positional Encoding



+ 30 >







Plan of Att...

Simplified...

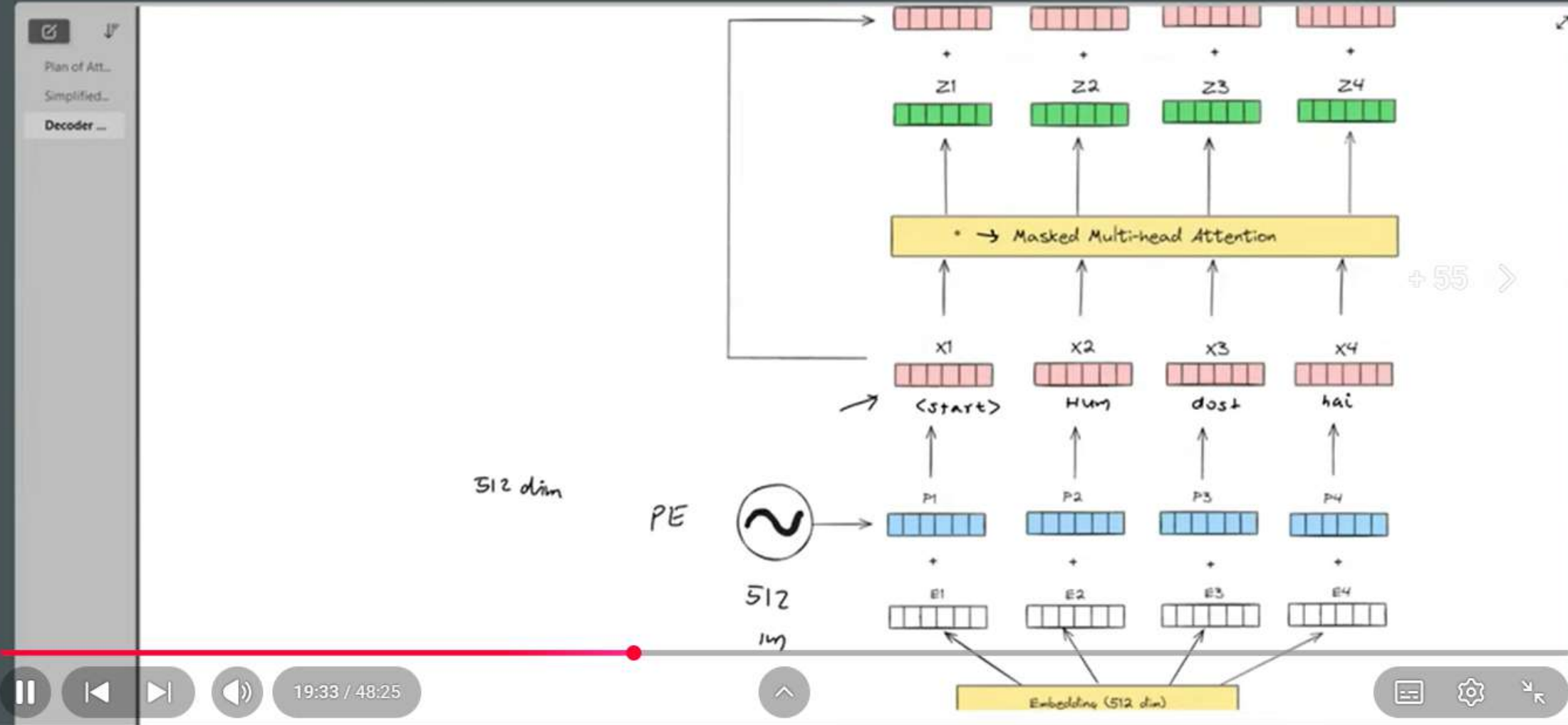
Decoder ...

1. Shifting
2. Tokenization
3. Embedding
4. Positional Encoding

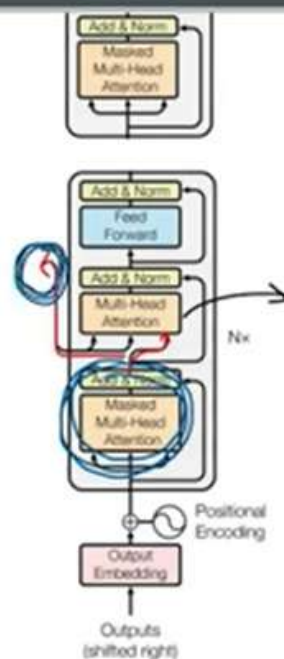
→

+ 30 ▶

Video player controls: Play/Pause, Previous, Next, Volume, Progress bar (18:44 / 48:25), Full Screen, Settings, and a close button.







cross attention

1 seq
2 seq

1. Shifting
2. Tokenization
3. Embedding
4. Positional Encoding

encoder
K V

hindi (Q)
[dewdu]

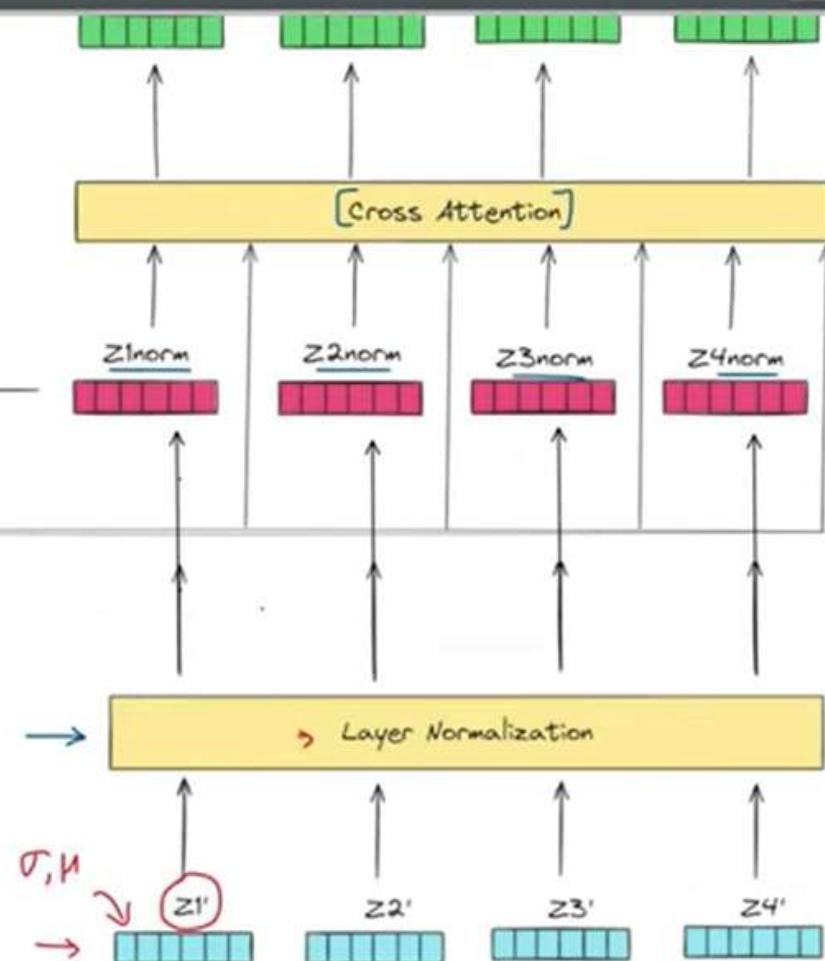


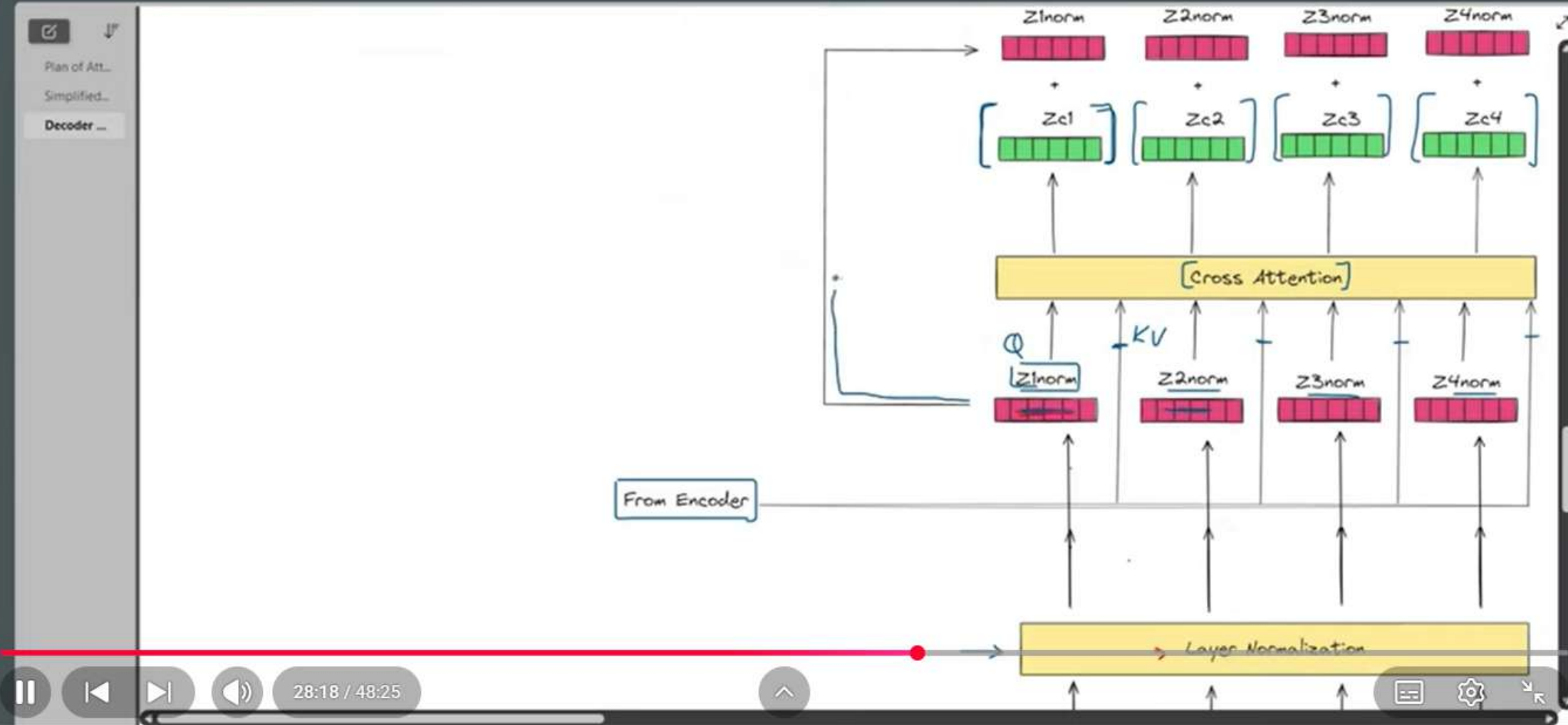
Plan of Att...

Simplified...

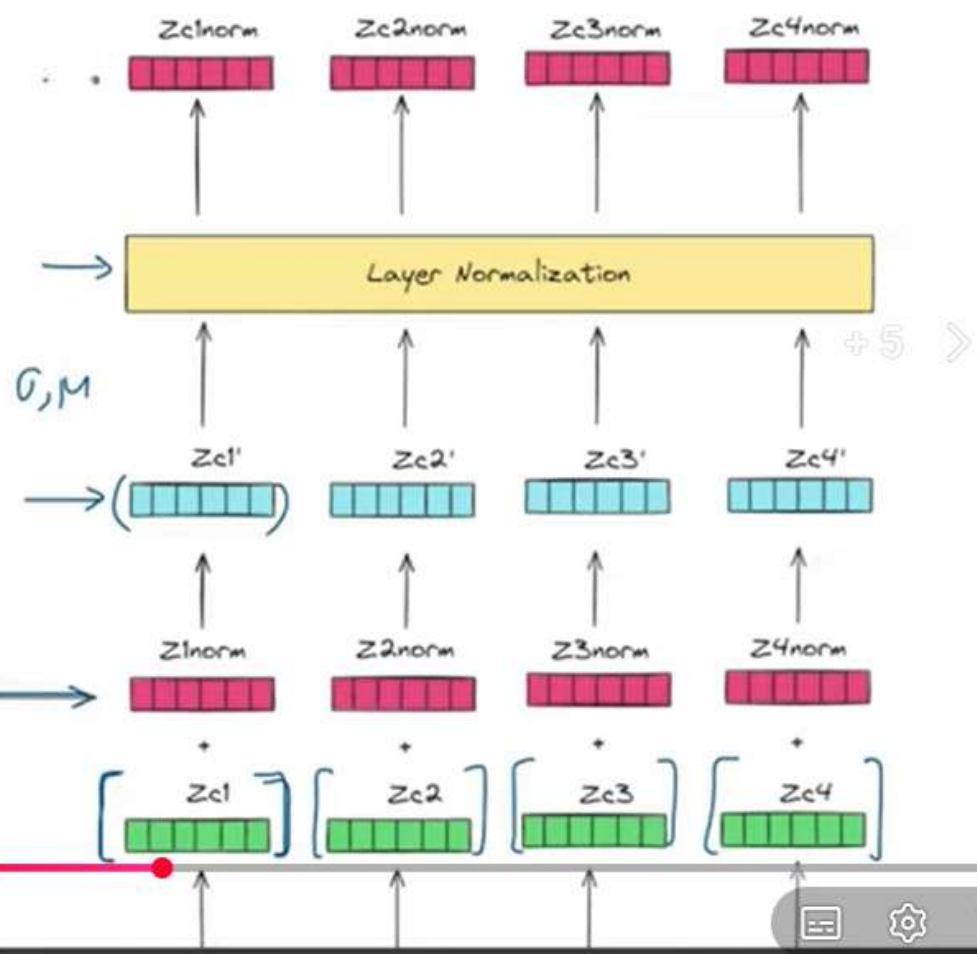
Decoder ...

From Encoder





- Plan of Att...
- Simplified...
- Decoder ...

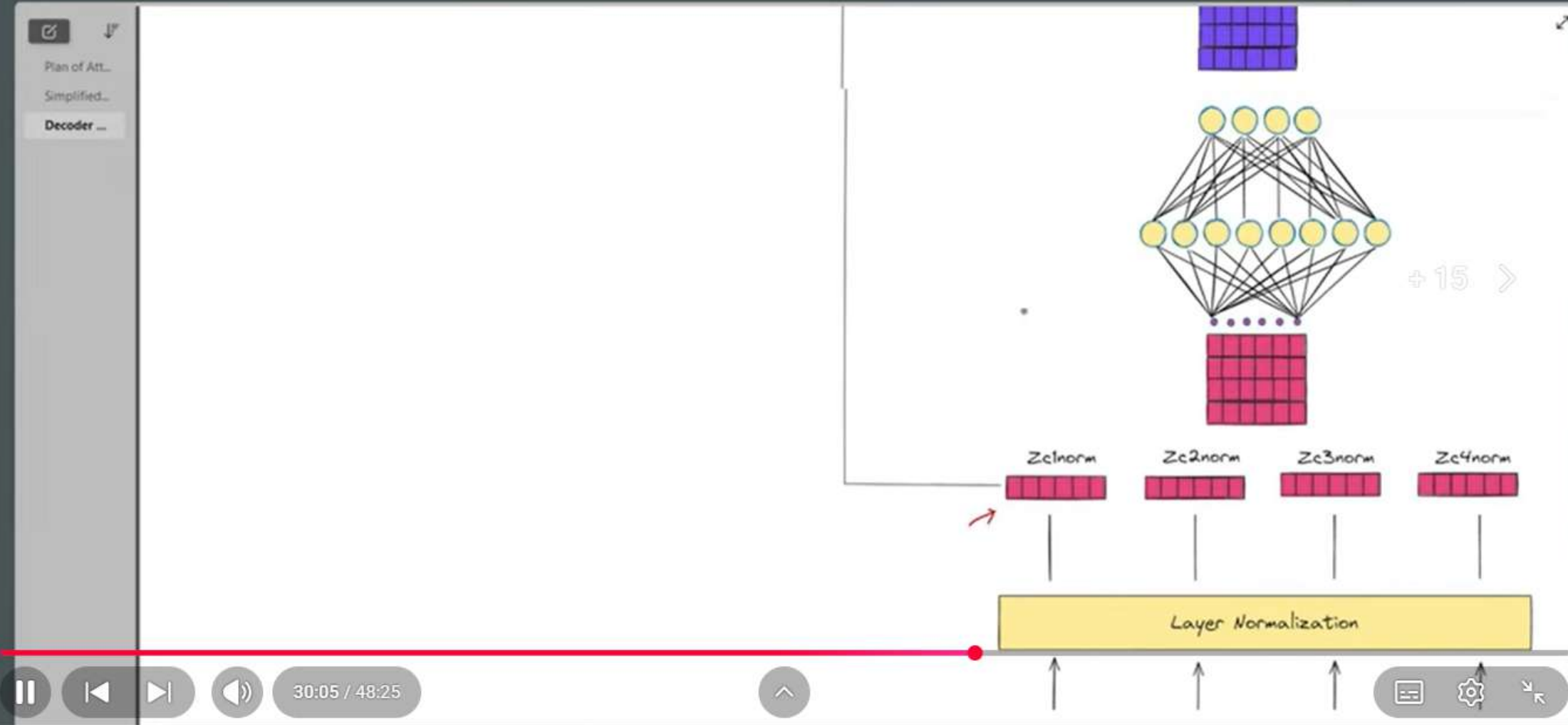


Plan of Att...

Simplified...

Decoder ...

1. Shifting
2. Tokenization
3. Embedding
4. Positional Encoding



Plan of Att...
Simplified...
Decoder...

2 layers

2048 neurons $\rightarrow \text{relu}$

512 neurons $\rightarrow \text{linear}$

4×512 512

4×2048 2048×512

4×512 512×2048 $\left[\text{relu}(W_1 Z + b_1) \right] W_2$

$4 \times 2048 + 2048$

512×2048

$512 \rightarrow$

$2048 \rightarrow$

$512 \rightarrow$

$512 (b_2)$

$2048 \times 512 (W_2)$

$2048 (b_1)$

W_1

4×512

4×2048

$Zc1norm$

$Zc2norm$

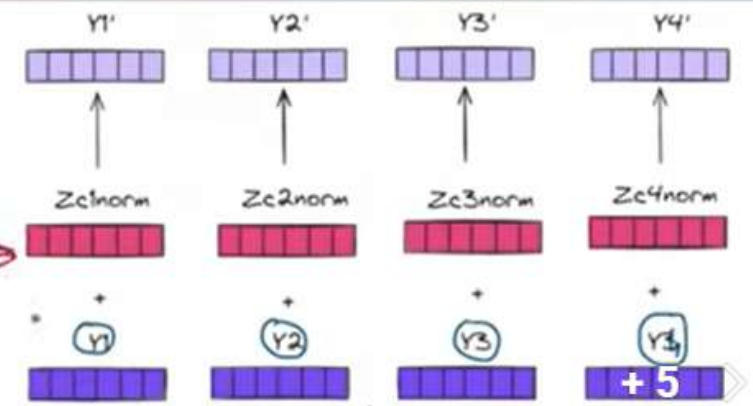
$Zc3norm$

$Zc4norm$

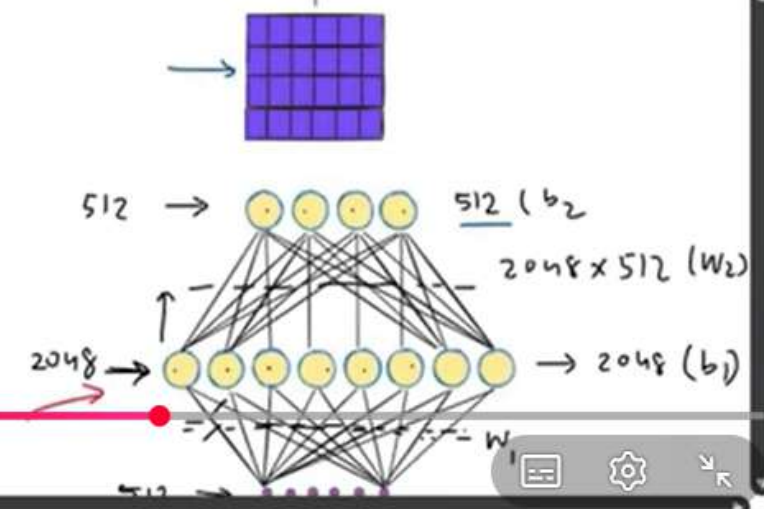
4. Positional Encoding

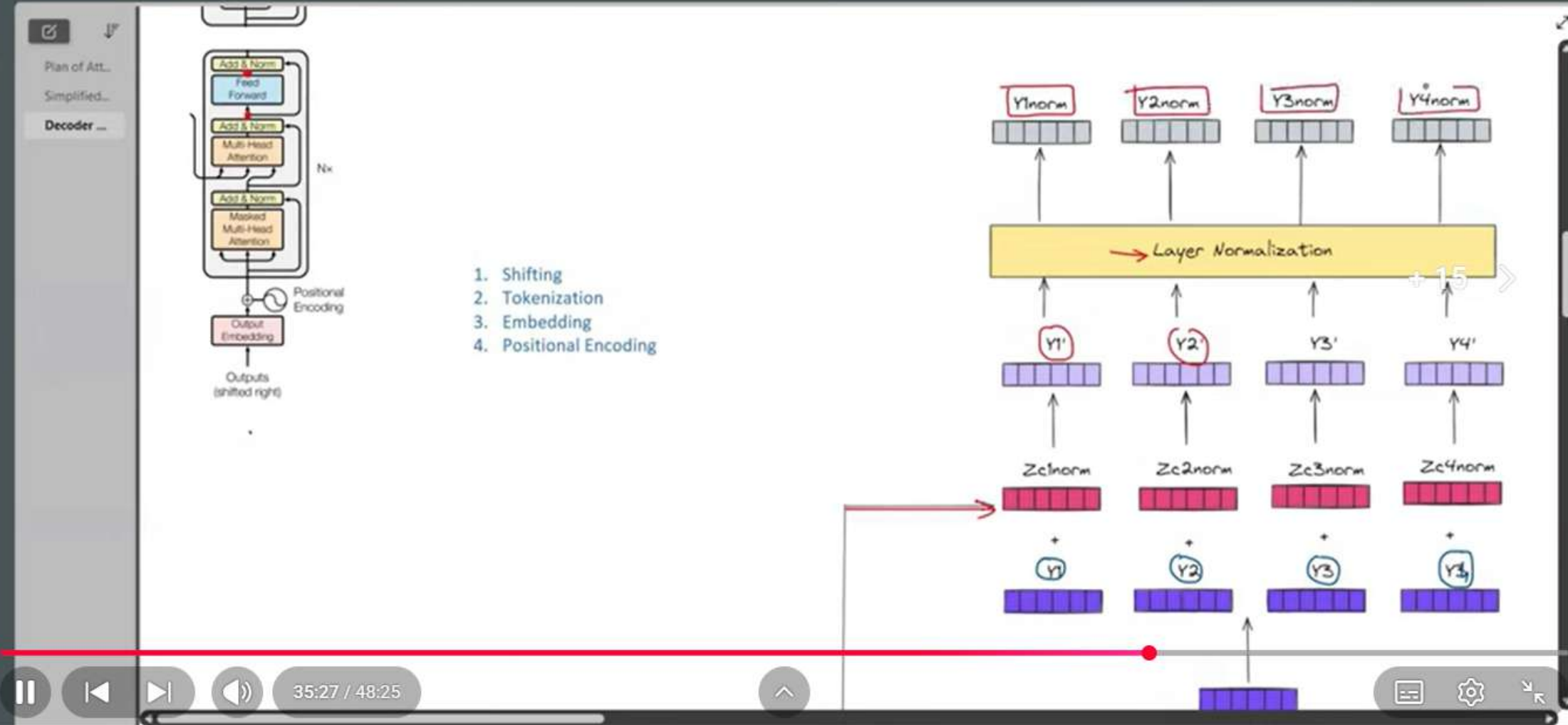
Embedding
Outputs
(shifted right)

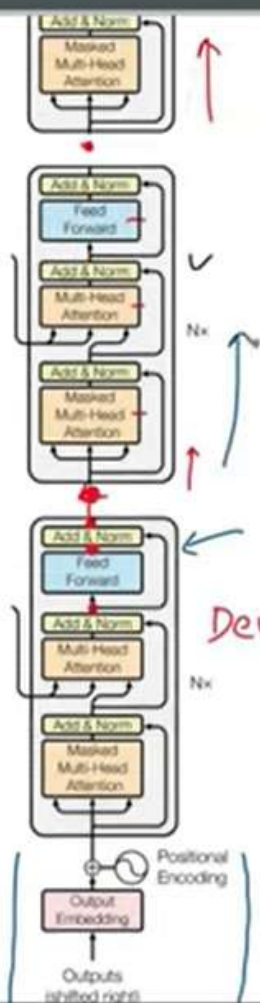
Plan of Att...
Simplified...
Decoder...



2 layers
2048 neurons $\rightarrow$ relu
 4×512 512
 $[4 \times 512]$
 512 neurons $\rightarrow$ linear
layer 2048×512



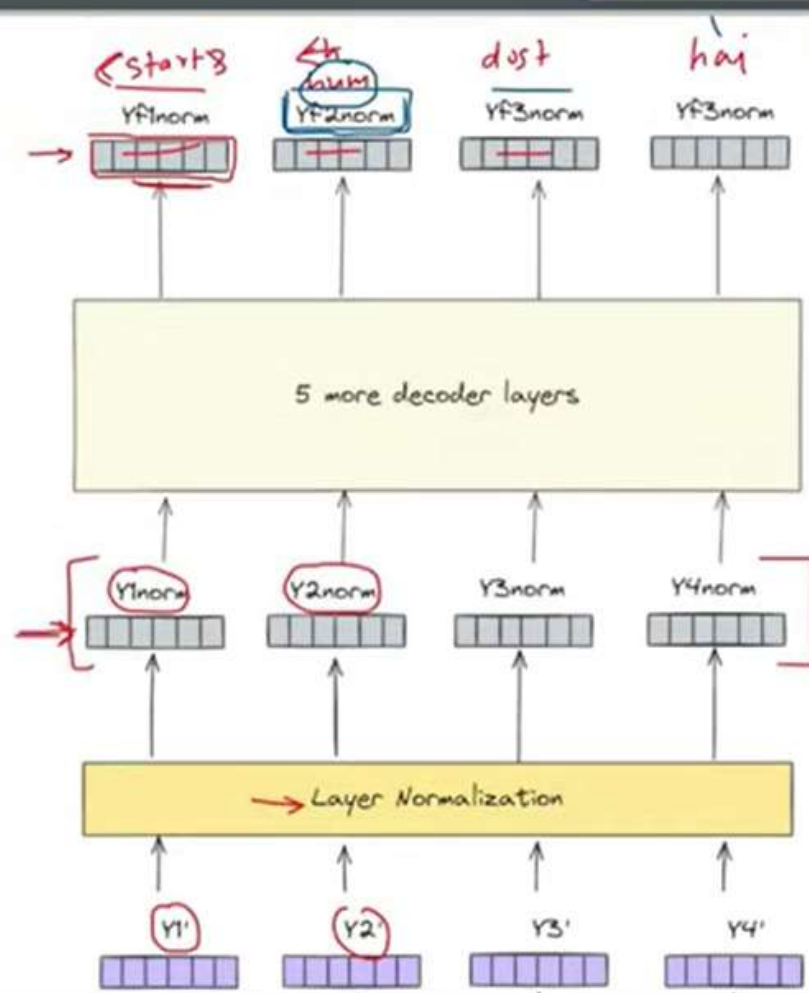


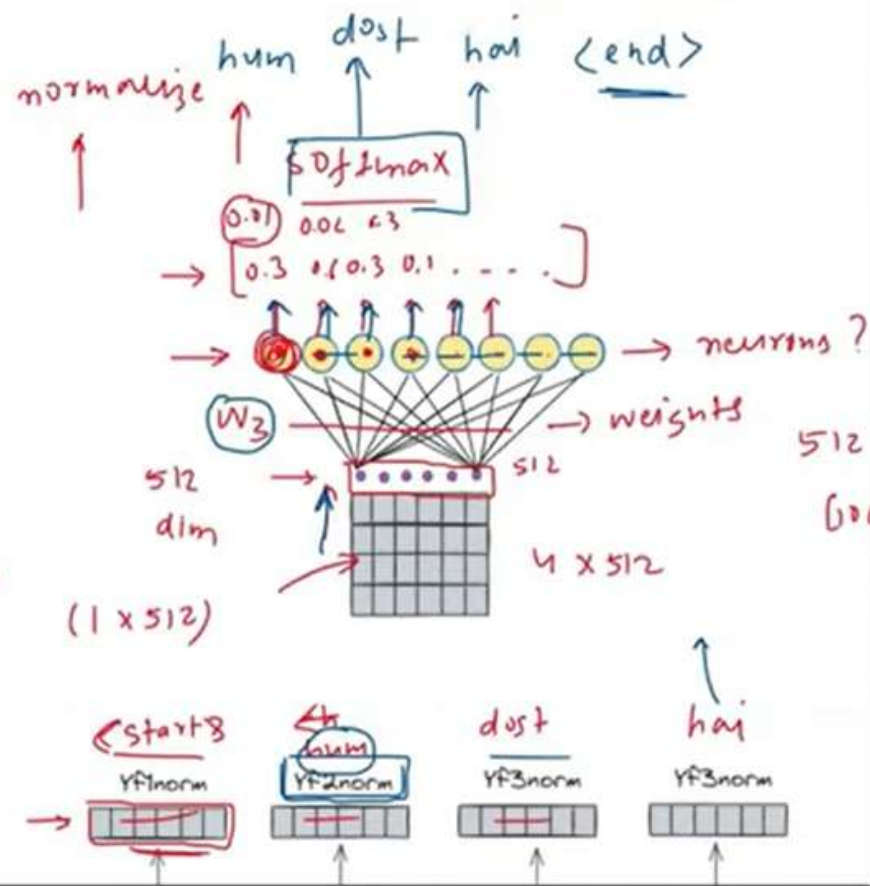


Decoder 1

1. Shifting
2. Tokenization
3. Embedding
4. Positional Encoding

1x10000





Transformer during Inference

03 September 2024 00:50

